AWS VPC Networking Lab

Step-by-Step Documentation: Creating a VPC, Subnets, and Internet Gateway

Prepared from the AWS Management Console screenshots

22nd October 2026

1 Description

This lab documents the setup of an AWS Virtual Private Cloud (VPC), a public subnet, a private subnet, and an Internet Gateway. Later phases will cover route tables, EC2 instances, and connectivity between the instances.

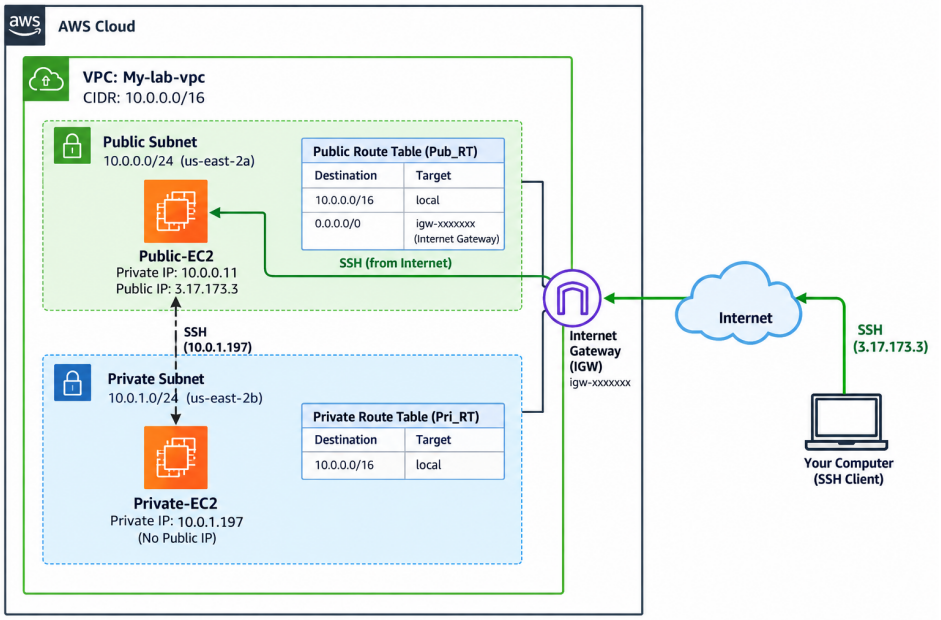


Figure 1: AWS VPC topology diagram.

2 Objective and Lab Overview

This document records the first phase of the AWS networking lab, from creating a custom Virtual Private Cloud (VPC) through creating public and private subnets and attaching an Internet Gateway (IGW). The next phase will document route-table configuration, EC2 instance creation, and connecting to both instances when the remaining screenshots and steps are provided.

2.1 Resources created in this phase

- VPC name: `My-lab-vpc`
- VPC IPv4 CIDR: `10.0.0.0/16`
- Public subnet: `Public-Subnet`, CIDR `10.0.0.0/24`, Availability Zone `us-east-2a`

- Private subnet: `Private_Subnet`, CIDR `10.0.1.0/24`, Availability Zone `us-east-2b`
- Internet Gateway: `My-Lab-IGW`, attached to `My-lab-vpc`

The console is set to the AWS US East (Ohio) Region. The VPC uses the default tenancy setting, has no IPv6 CIDR block, and has VPC encryption control set to None.

3 Step 1: Create the VPC

Purpose: A VPC is an isolated virtual network in AWS. Its IPv4 CIDR block defines the private IPv4 address space available for its subnets.

1. Open the Amazon VPC console and choose **Your VPCs** → **Create VPC**.
2. Select **VPC only**. This creates just the VPC, without automatically creating subnets or other networking resources.
3. Enter `My-lab-vpc` as the Name tag.
4. Choose manual IPv4 CIDR input and enter `10.0.0.0/16`. This provides the VPC address range `10.0.0.0`–`10.0.255.255`.
5. Select **No IPv6 CIDR block**; leave tenancy as **Default** and VPC encryption control as **None** for this lab.
6. Choose **Create VPC** and verify the success message.

Important: Creating a VPC does not automatically make it accessible from the internet. It initially has no subnets in this custom VPC, and no Internet Gateway route has been configured.

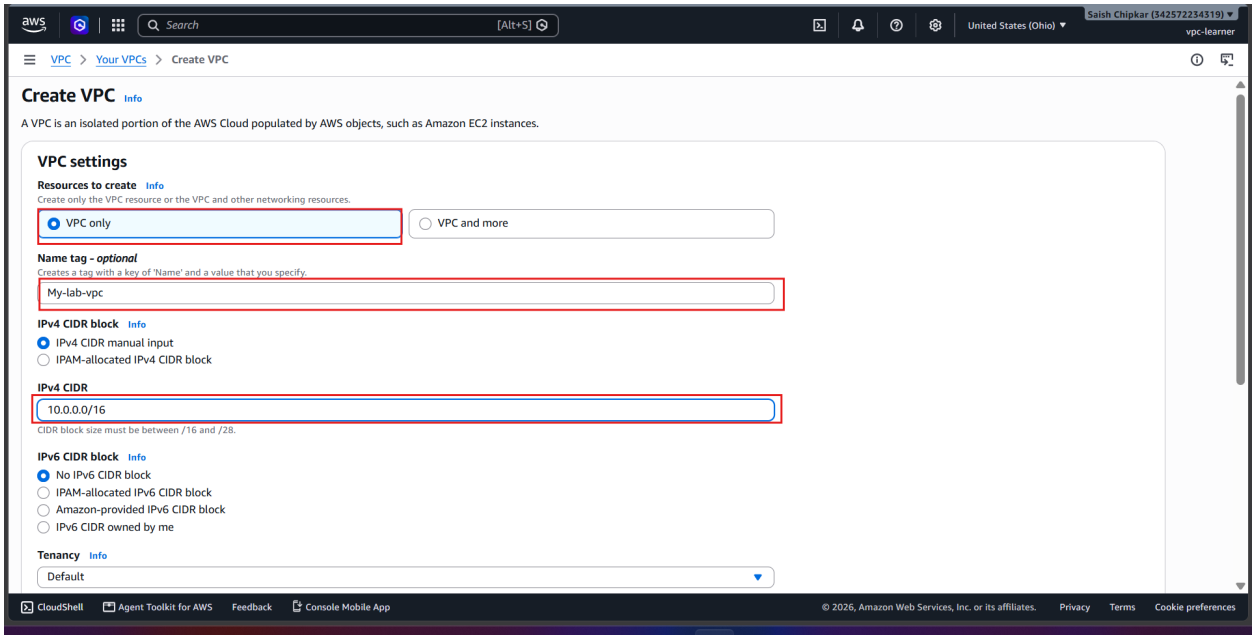


Figure 2: VPC creation form: VPC only, name, and IPv4 CIDR settings.

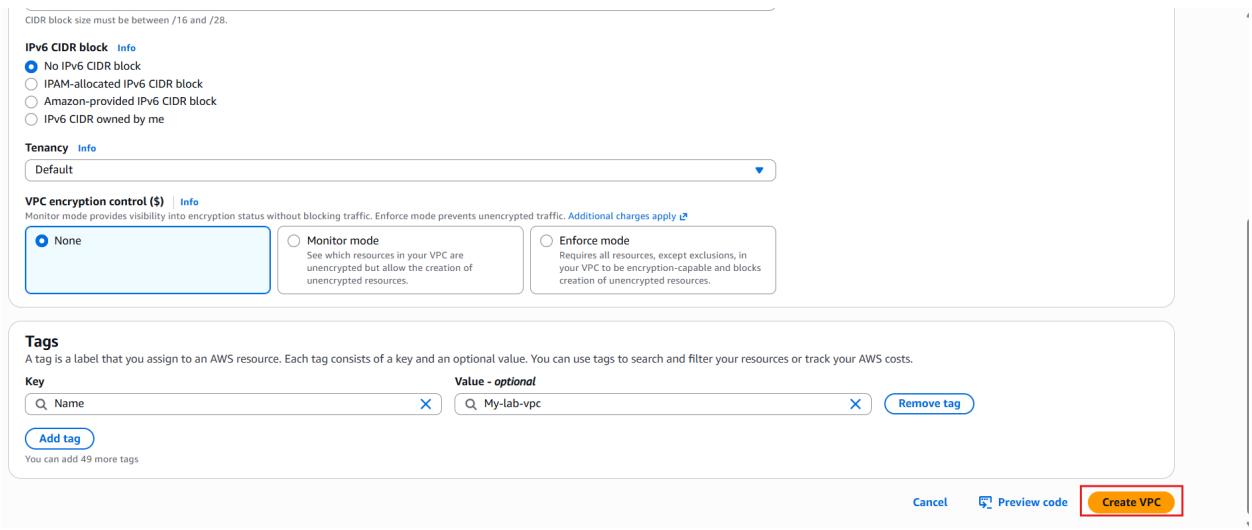


Figure 3: Remaining VPC settings and the Create VPC action.

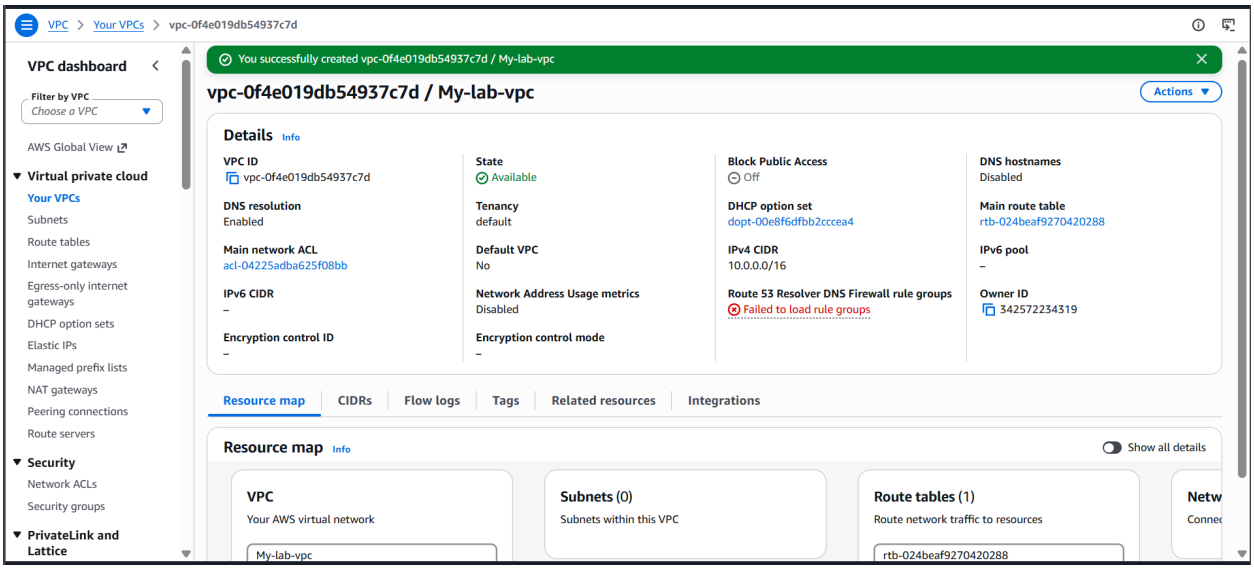


Figure 4: Successful creation of My-lab-vpc and its VPC details.

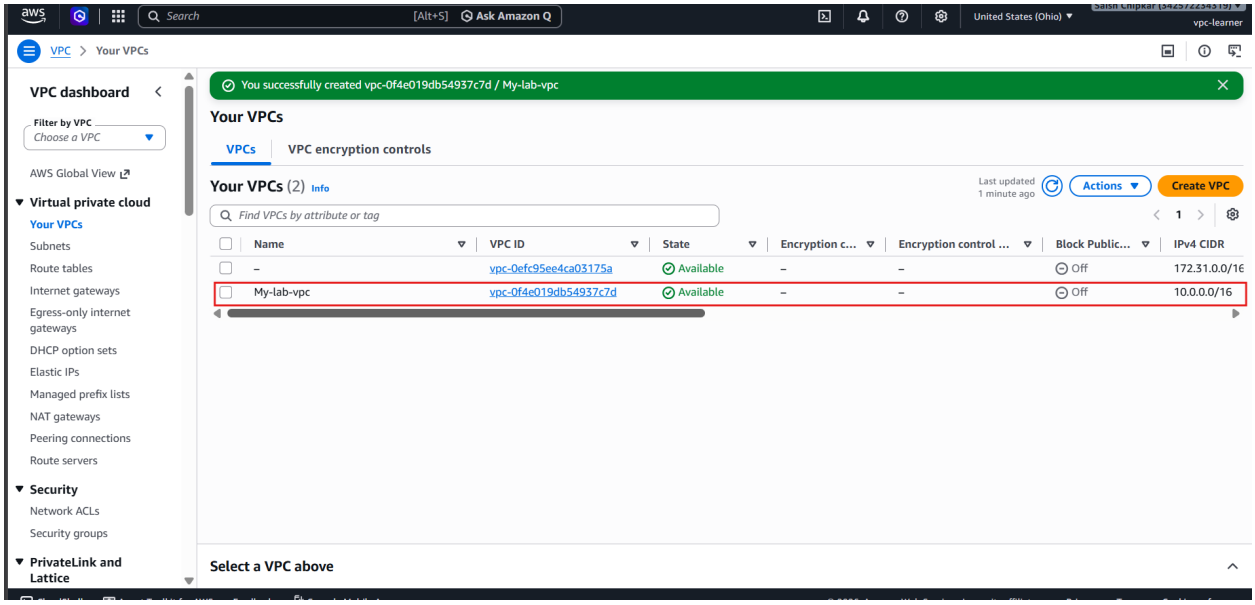


Figure 5: VPC list showing the newly created custom VPC.

4 Step 2: Create the Public Subnet

Purpose: A subnet is a smaller IP address range inside a VPC and exists in one Availability Zone. A subnet is not public merely because it is named "Public-Subnet"; its route table and the instance's public IP configuration also matter.

1. Open **Subnets** in the VPC console and choose **Create subnet**.
2. Select the custom VPC `My-lab-vpc` (CIDR `10.0.0.0/16`), not the default VPC.
3. Set the subnet name to `Public-Subnet`.
4. Choose Availability Zone `us-east-2a` (shown as use2-az1 in the console).
5. Select the VPC CIDR block `10.0.0.0/16` and enter subnet CIDR `10.0.0.0/24`.
6. Optionally add the tag `Type = Public`, then choose **Create subnet**.

A `/24` subnet contains 256 IPv4 addresses in total. AWS reserves five addresses in each IPv4 subnet, so 251 are shown as available in the console.

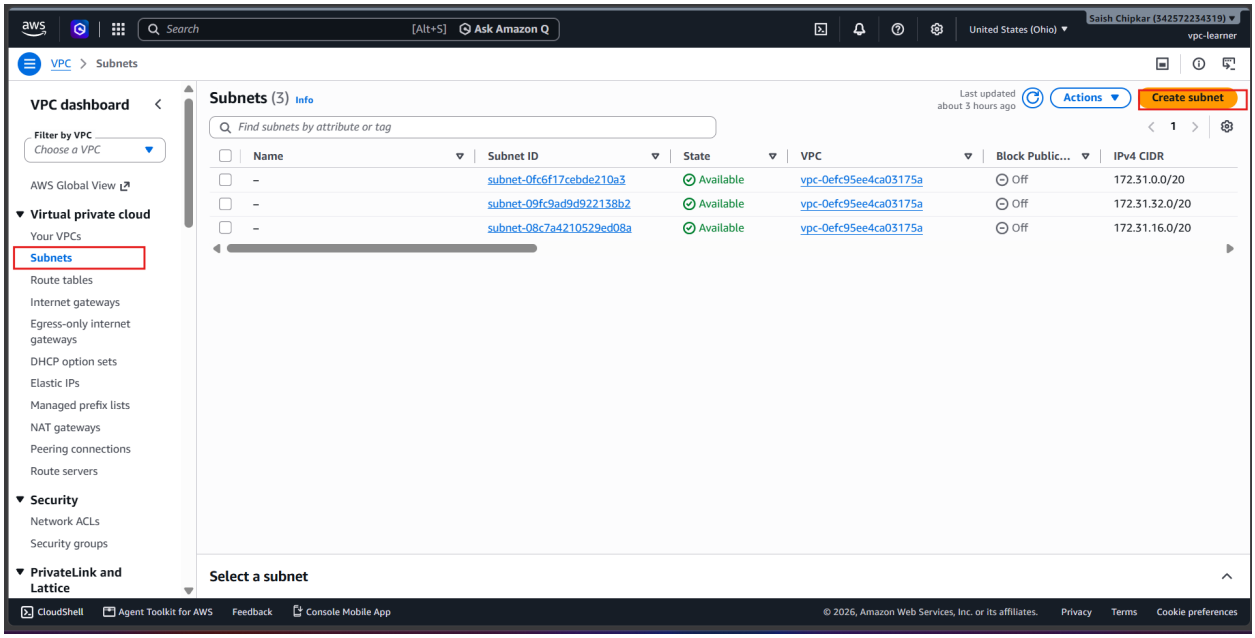


Figure 6: Open the Subnets page and begin creating a subnet.

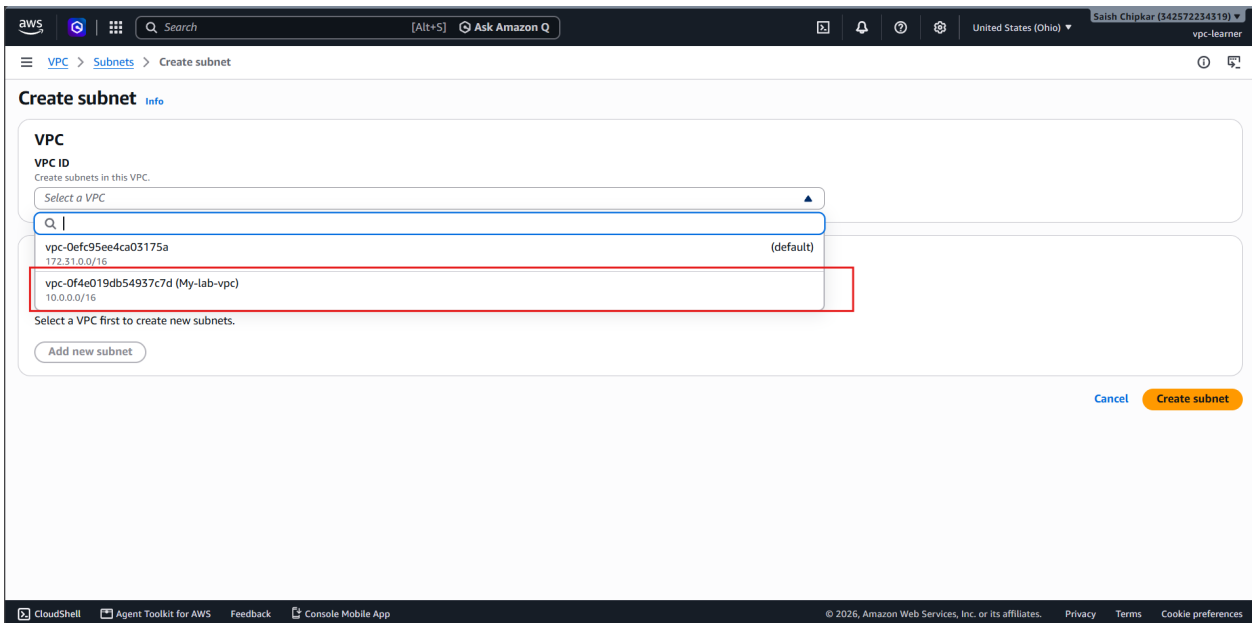


Figure 7: Choose My-lab-vpc in the VPC selector.

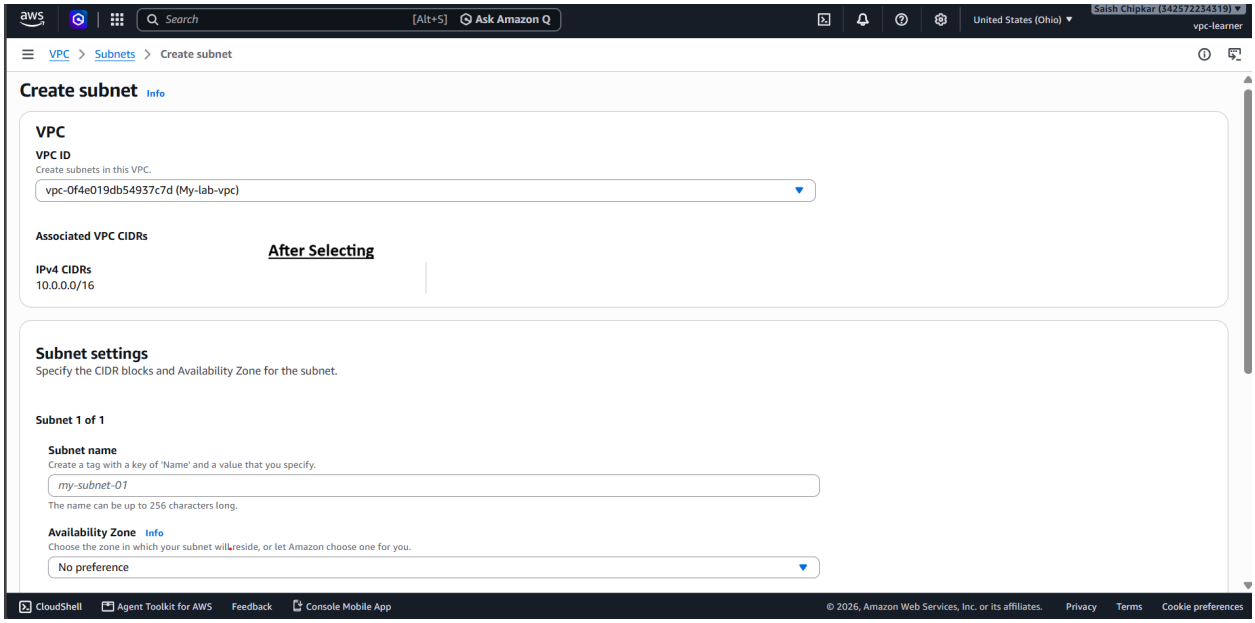


Figure 8: Subnet form after selecting My-lab-vpc; enter the public subnet settings.

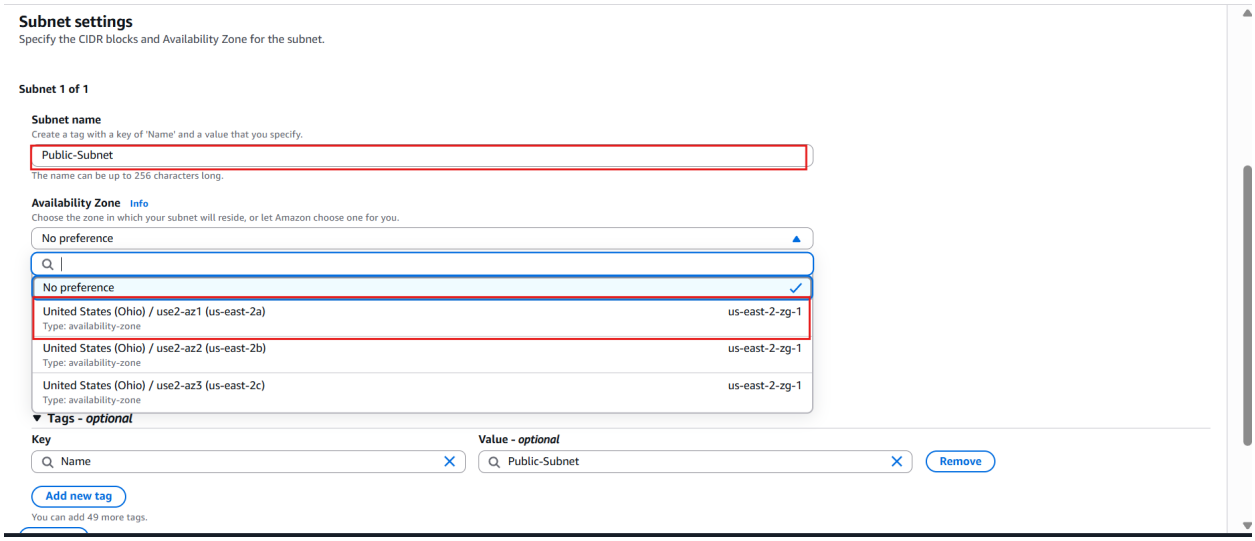


Figure 9: Set the public subnet name and select Availability Zone us-east-2a.

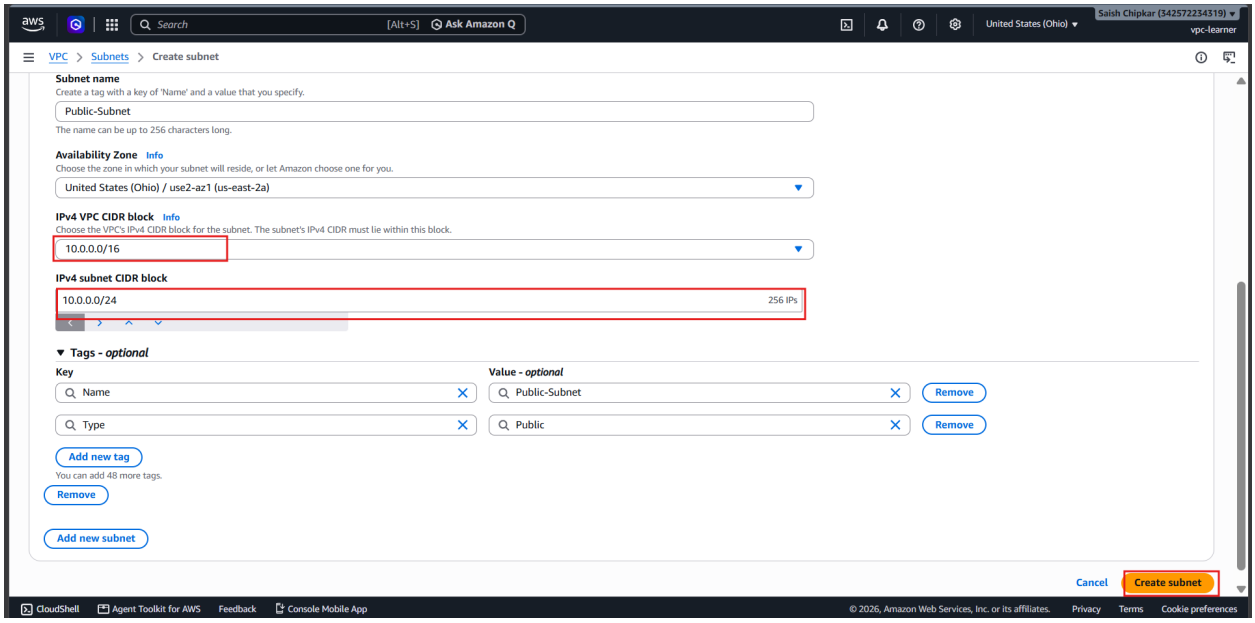


Figure 10: Configure the public subnet CIDR as 10.0.0.0/24 and create the subnet.

5 Step 3: Create the Private Subnet

Purpose: The private subnet provides a separate address range for resources that should not be directly reachable from the public internet.

1. From the Subnets page, choose **Create subnet** and again select `My-lab-vpc`.
2. Set the subnet name to `Private_Subnet`.
3. Choose Availability Zone `us-east-2b` (shown as use2-az2 in the console).
4. Set the subnet IPv4 CIDR to `10.0.1.0/24`. This does not overlap with the public subnet's `10.0.0.0/24` range.
5. Optionally add the tag `Type = Private`, then choose **Create subnet**.

The two subnets are in different Availability Zones. Their IP ranges are distinct parts of the VPC's larger `10.0.0.0/16` range.

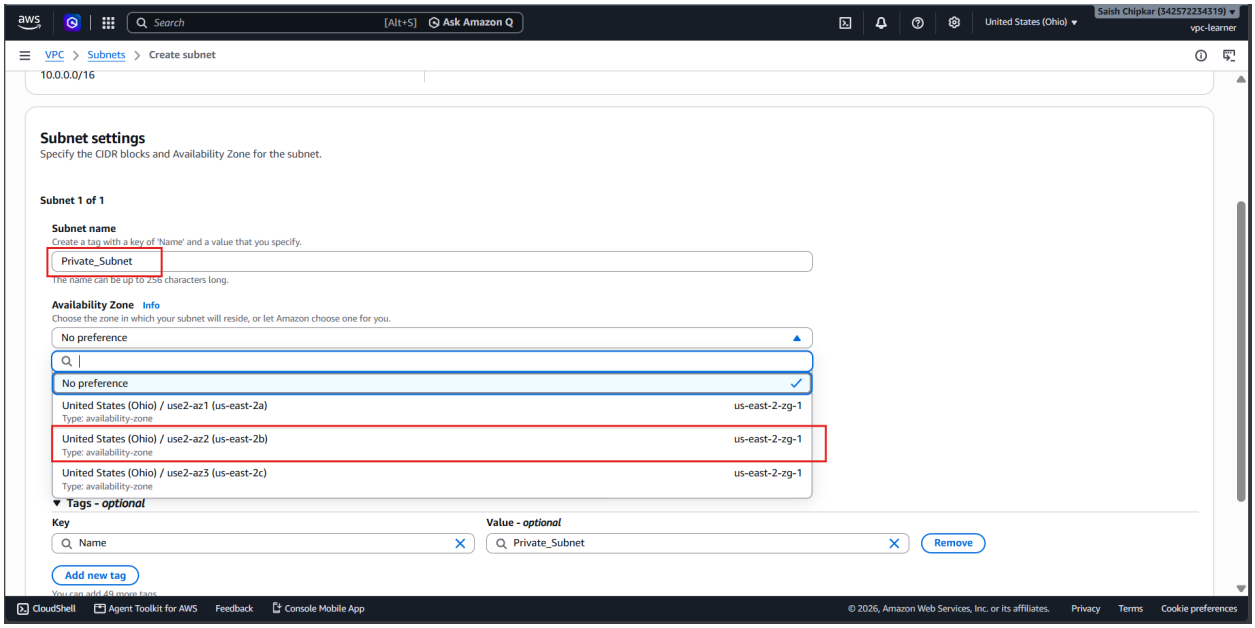


Figure 11: Set the private subnet name and choose Availability Zone us-east-2b.

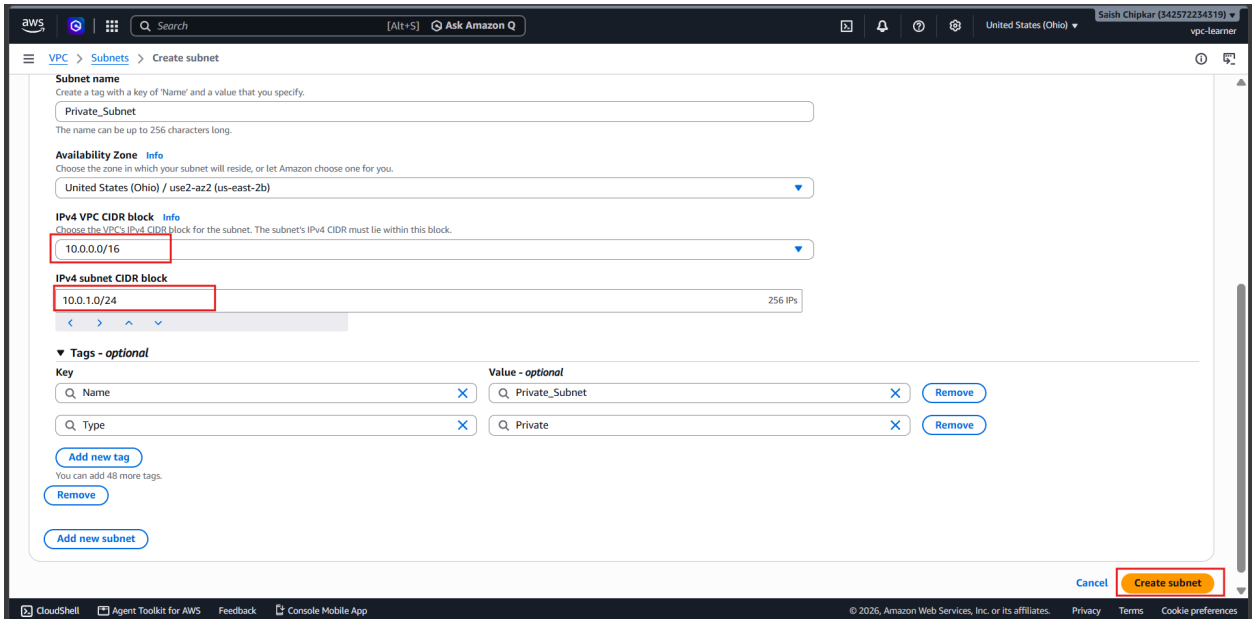


Figure 12: Configure the private subnet CIDR as 10.0.1.0/24 and create it.

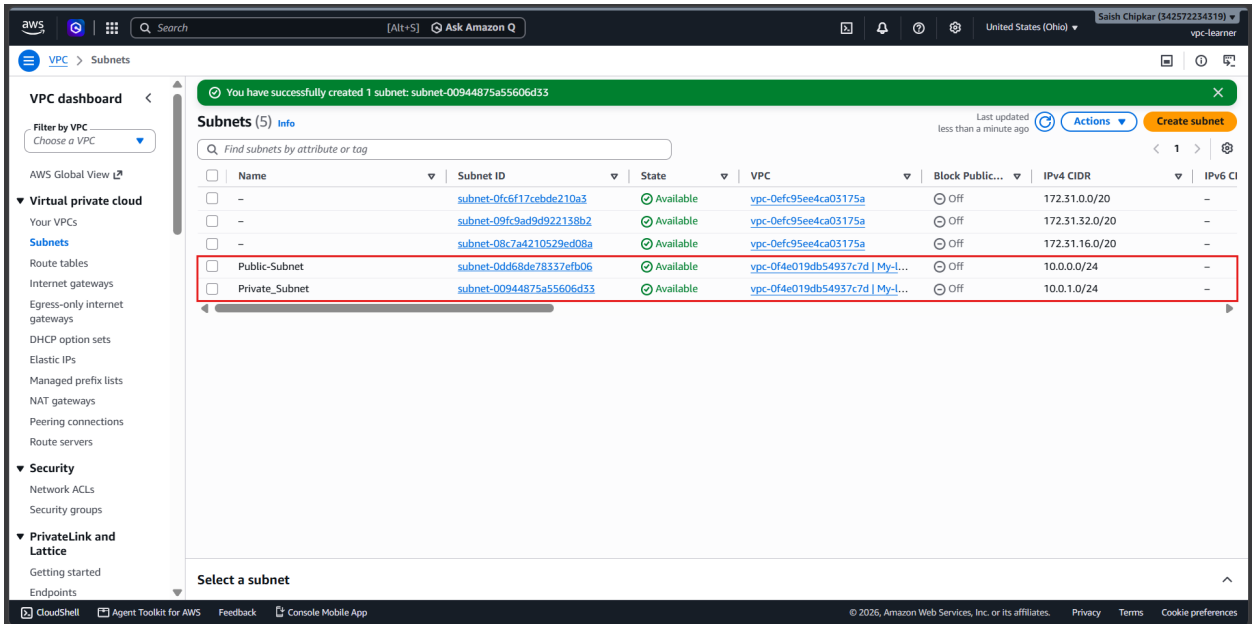


Figure 13: Subnets list showing the public and private subnets created in My-lab-vpc.

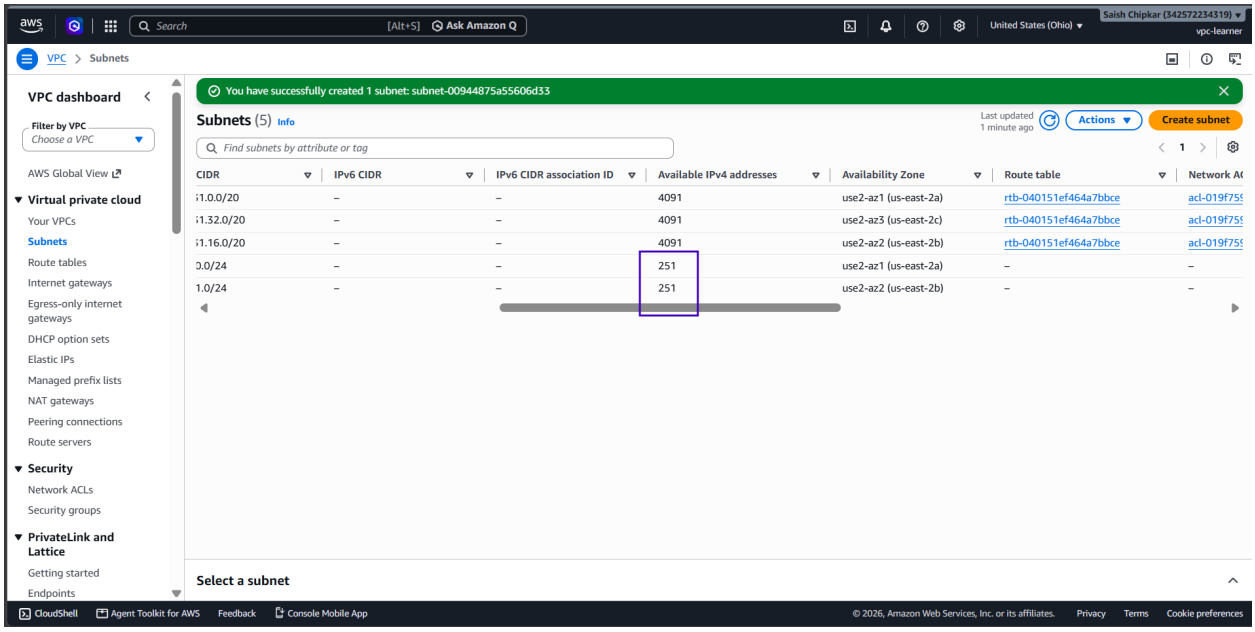


Figure 14: Subnet table details, including available IPv4 addresses and Availability Zones.

Note: Each `/24` subnet has 256 total IPv4 addresses. AWS reserves these five addresses in each subnet, leaving 251 available:

Public subnet	**Private subnet**	**Reserved purpose**
`10.0.0.0`	`10.0.1.0`	Network address
`10.0.0.1`	`10.0.1.1`	VPC router
`10.0.0.2`	`10.0.1.2`	DNS server
`10.0.0.3`	`10.0.1.3`	Reserved for future use
`10.0.0.255`	`10.0.1.255`	Broadcast address (reserved by AWS)

Across both subnets, 512 addresses exist in total and 502 are available for allocation.

6 Step 4: Create an Internet Gateway

Purpose: An Internet Gateway is a VPC-level networking component that allows eligible resources in the VPC to communicate with the internet when routing and addressing requirements are also satisfied.

1. In the VPC console, select **Internet gateways**.
2. Choose **Create internet gateway**.
3. Enter the name tag `My-Lab-IGW` and choose **Create internet gateway**.
4. The new gateway initially shows the state **Detached**; it has not yet been associated with a VPC.

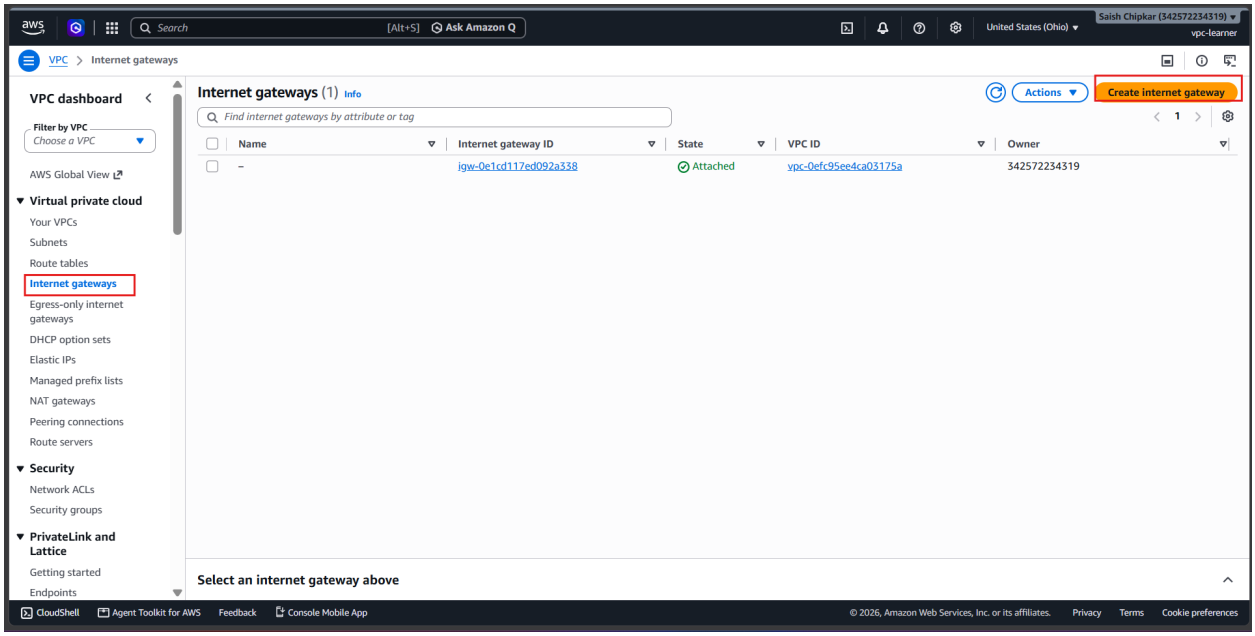


Figure 15: Open the Internet gateways page and choose Create internet gateway.

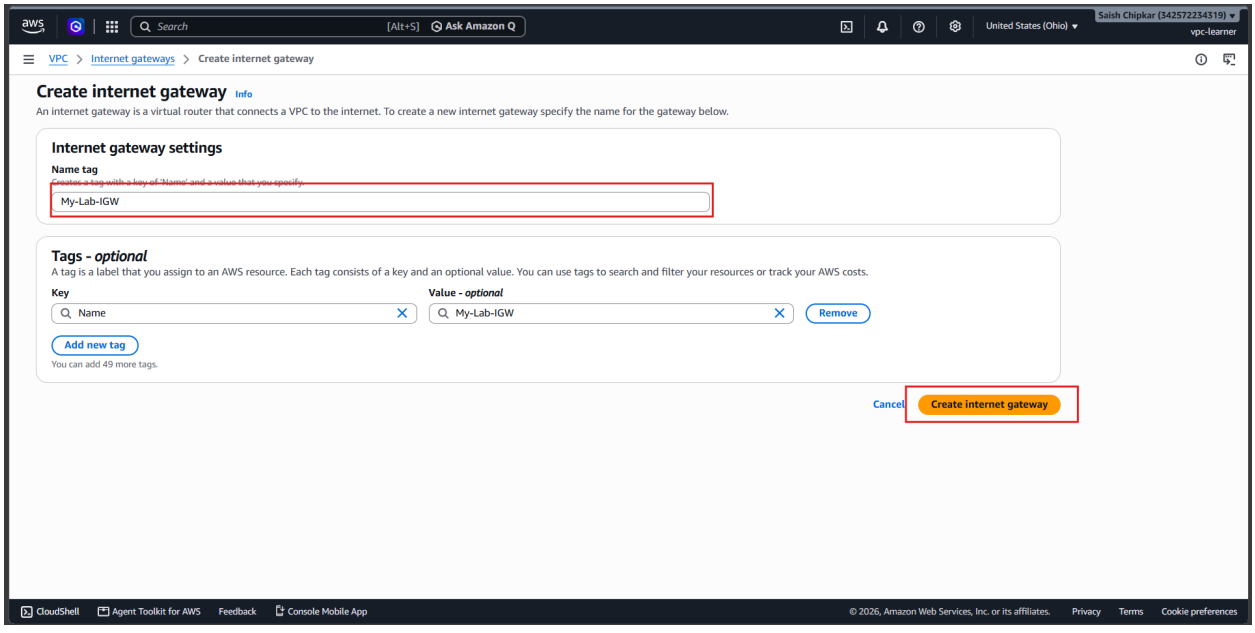


Figure 16: Enter the Internet Gateway name tag.

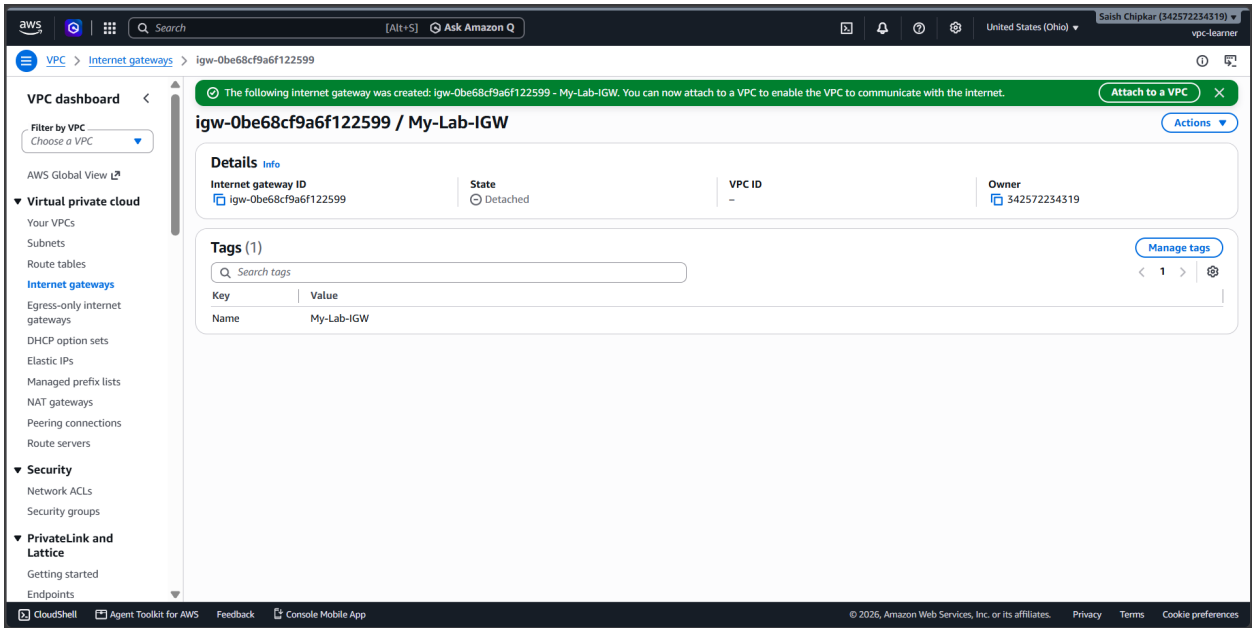


Figure 17: New Internet Gateway created in Detached state.

7 Step 5: Attach the Internet Gateway to My-lab-vpc

1. Select `My-Lab-IGW` in the Internet gateways list.
2. Choose **Actions** → **Attach to VPC** (or use the Attach to a VPC button).
3. Select `vpc-0f4e019db54937c7d (My-lab-vpc)`.
4. Choose **Attach internet gateway**.
5. Verify that the state is **Attached** and that the VPC ID is populated.

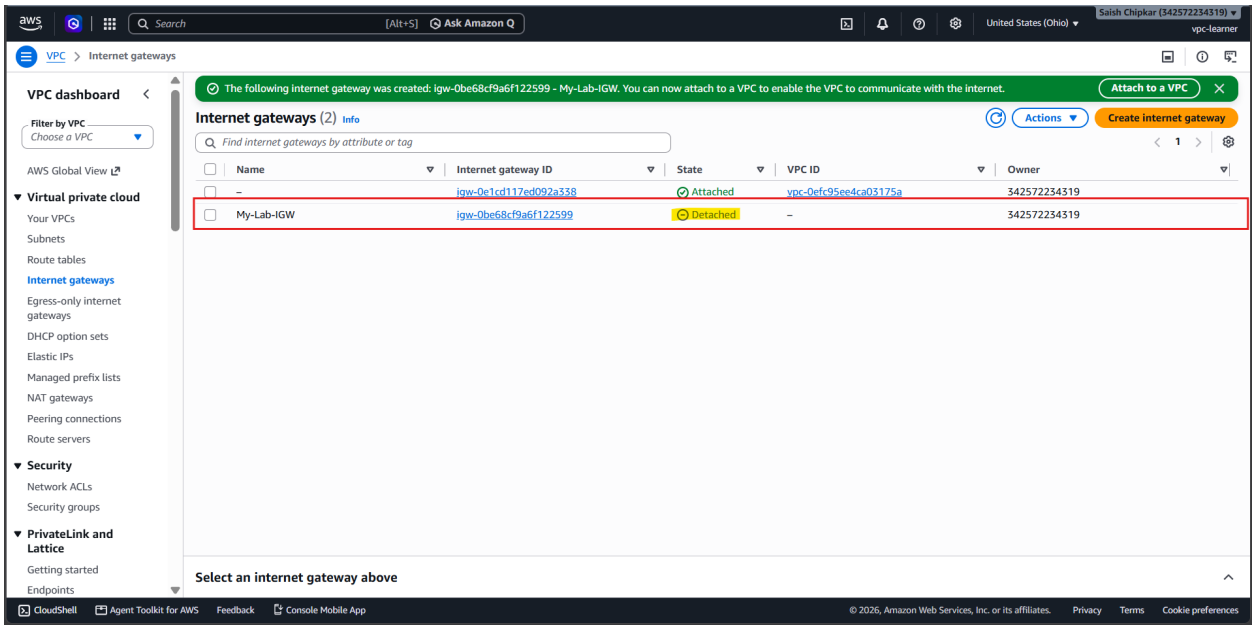


Figure 18: Internet Gateway list showing My-Lab-IGW in the Detached state.

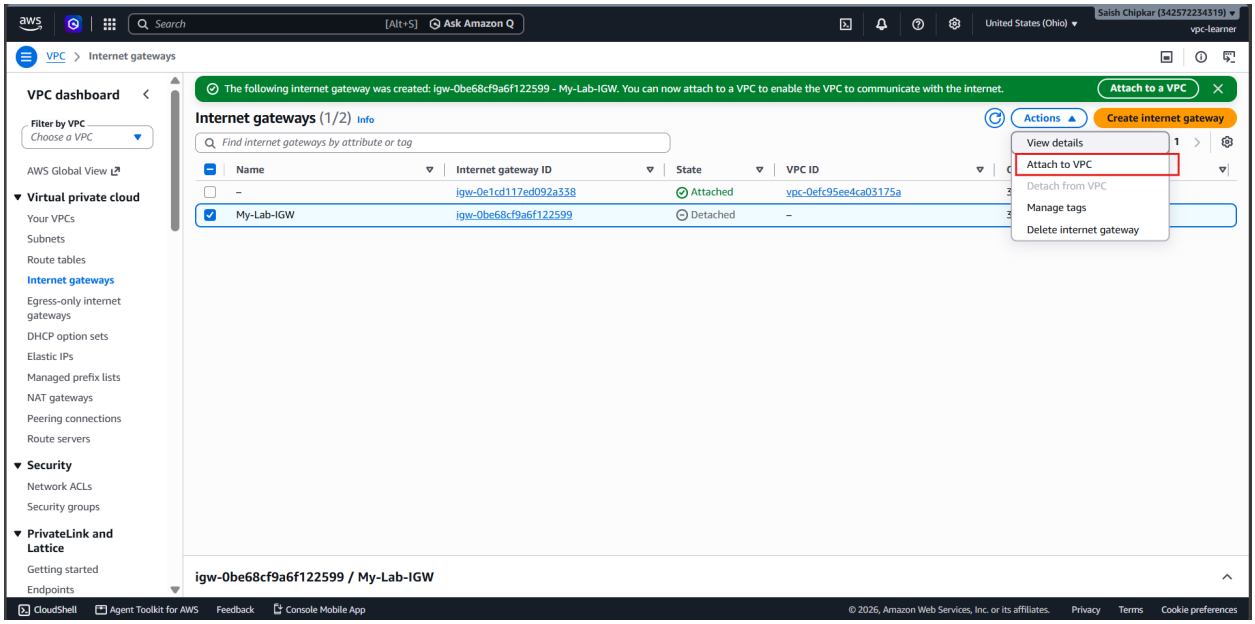


Figure 19: Open Actions and choose Attach to VPC.

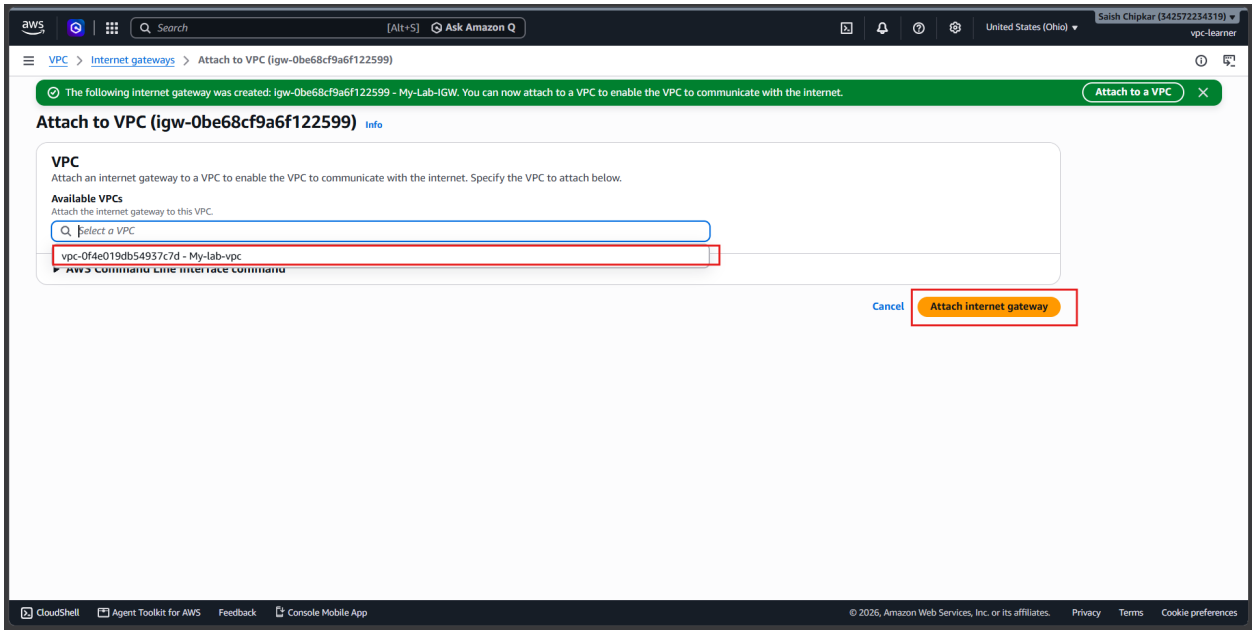


Figure 20: Choose My-lab-vpc as the VPC to attach the gateway to.

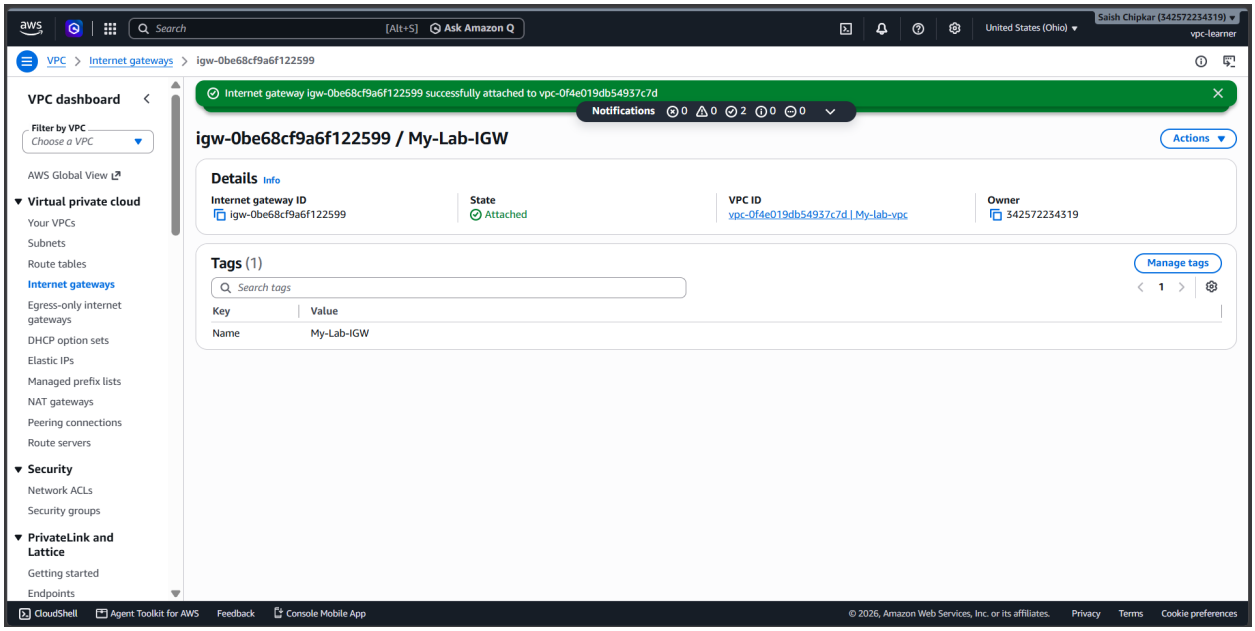


Figure 21: Successful attachment of My-Lab-IGW to My-lab-vpc.

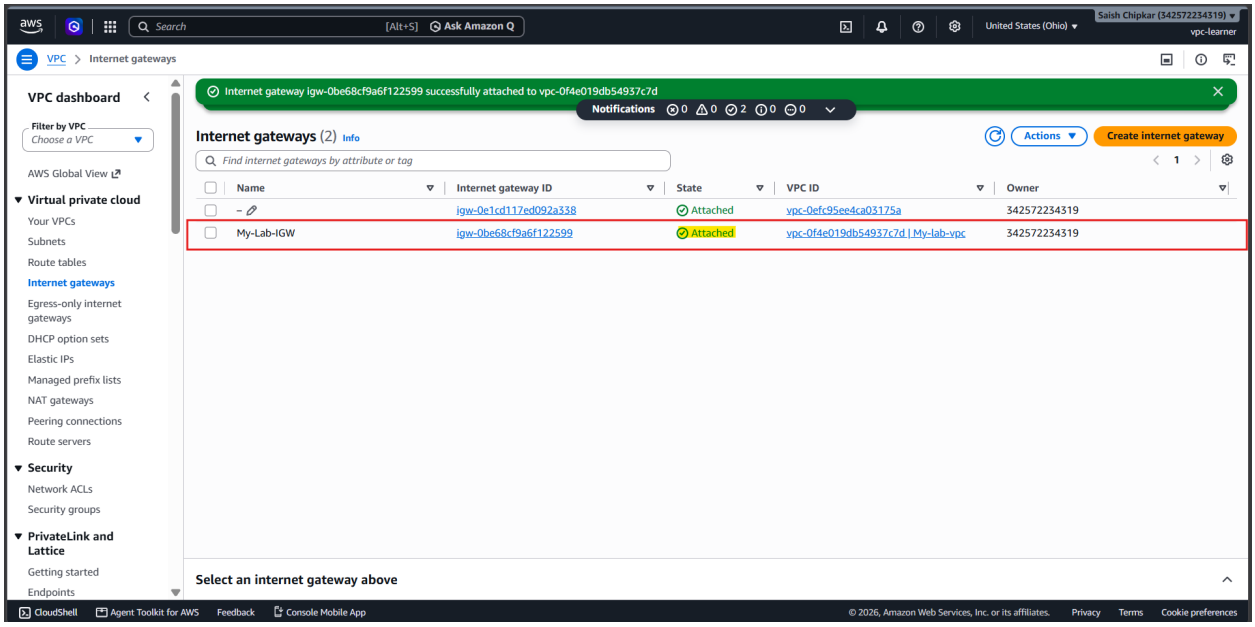


Figure 22: Internet Gateway page showing My-Lab-IGW attached to My-lab-vpc.

8 Step 6: Create and Configure Route Tables

Purpose: A route table contains rules that determine where network traffic from a subnet is directed. In this lab, two separate route tables are created: one for the public subnet and one for the private subnet.

The public route table will contain a route to the Internet Gateway, whereas the private route table will retain only the local route to the VPC. Each route table will then be associated with its respective subnet.

8.1 Understanding the Existing Route Table

Before creating new route tables, the Route Tables page may already display a route table. This is normal in AWS.

AWS provides a **default VPC** in the account and automatically creates a main route table for it. The screenshot `22-N.png` shows the existing route table associated with the default VPC `vpc-0efc95ee4ca03175a`. This is different from the custom VPC created for this lab, `My-lab-vpc` (`vpc-0f4e019db54937c7d`).

AWS also automatically creates a main route table when a new custom VPC is created. Therefore, the route table that already exists does not need to be deleted or replaced.

What is the main route table?

The main route table is the default route table for a VPC. Any subnet that is not explicitly associated with another route table uses the main route table.

A newly created VPC's main route table automatically contains a local route. For this lab, the route is:

Destination	Target	Purpose
`10.0.0.0/16`	`local`	Allows communication between resources within the VPC's IPv4 address range, subject to network and security rules.

The `local` target is not an Internet Gateway. It represents routing within the VPC. AWS creates this route automatically, and it should normally be retained.

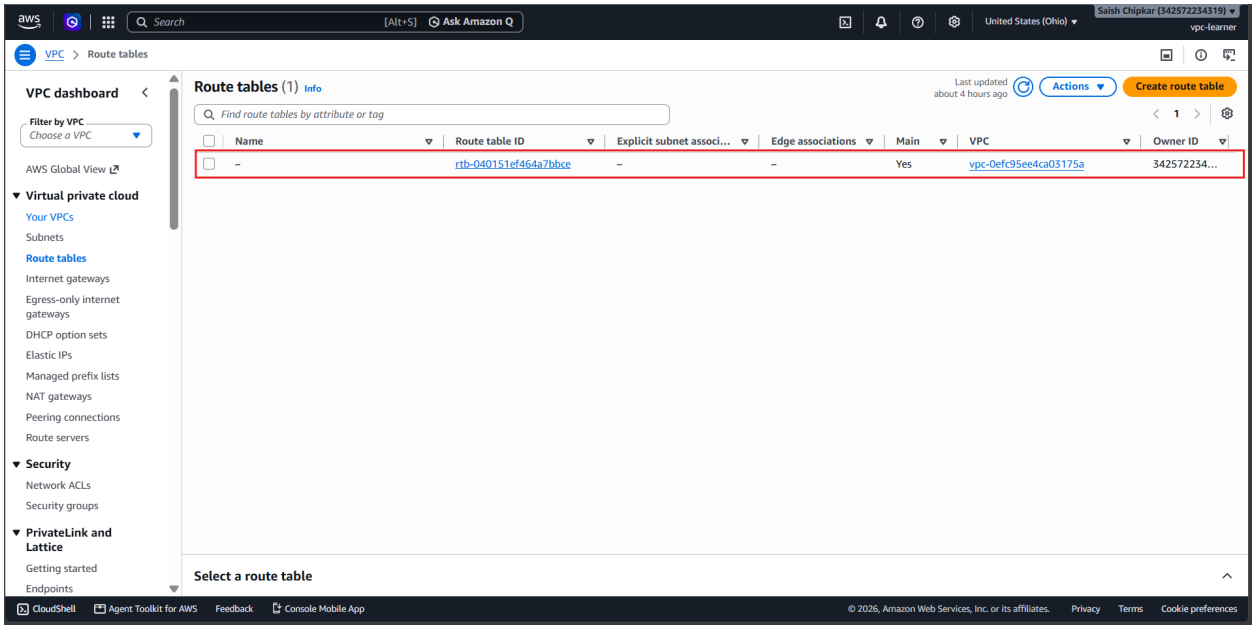


Figure 23: An existing route table associated with the default VPC, rather than the custom VPC used in this lab.

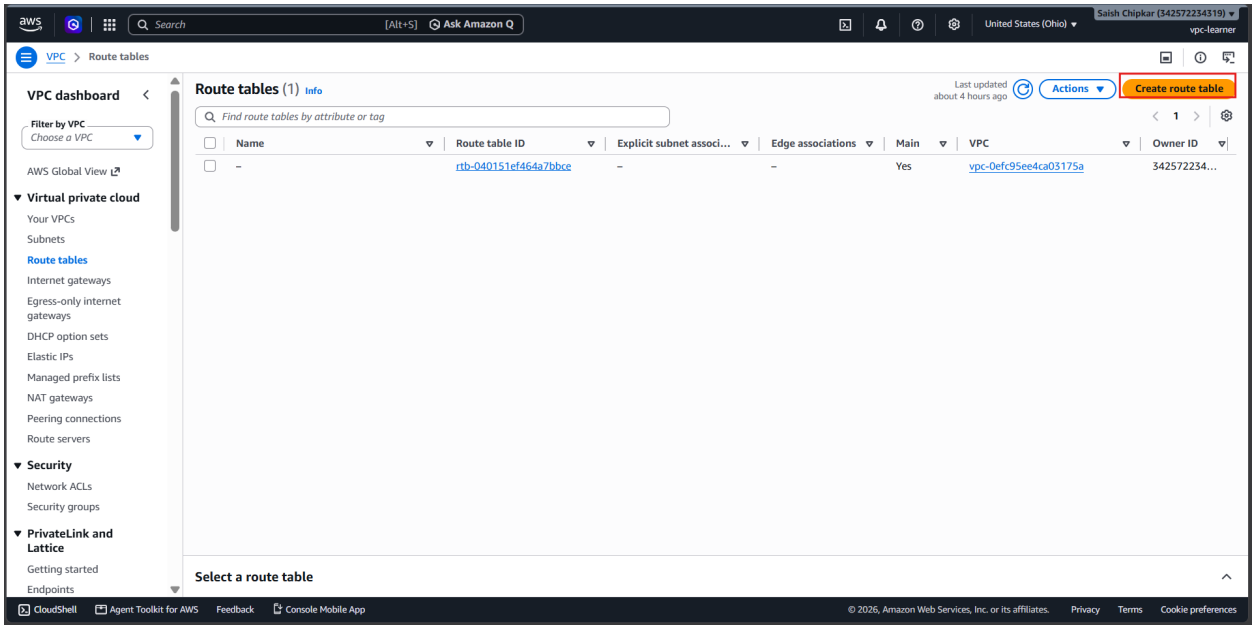


Figure 24: Route Tables page before creating the dedicated route tables for the custom VPC.

8.2 Create the Public Route Table

The public route table is created to direct internet-bound traffic from the public subnet to the Internet Gateway.

1. Open the Amazon VPC console and select **Route tables** from the navigation panel.
2. Choose **Create route table**.
3. Enter `Public-RT` in the Name field.
4. Under VPC, select `My-lab-vpc (10.0.0.0/16)`.
5. Choose **Create route table**.

After creation, AWS automatically adds the local route `10.0.0.0/16` with target `local`. At this stage, the route table does not yet contain a route to the internet.

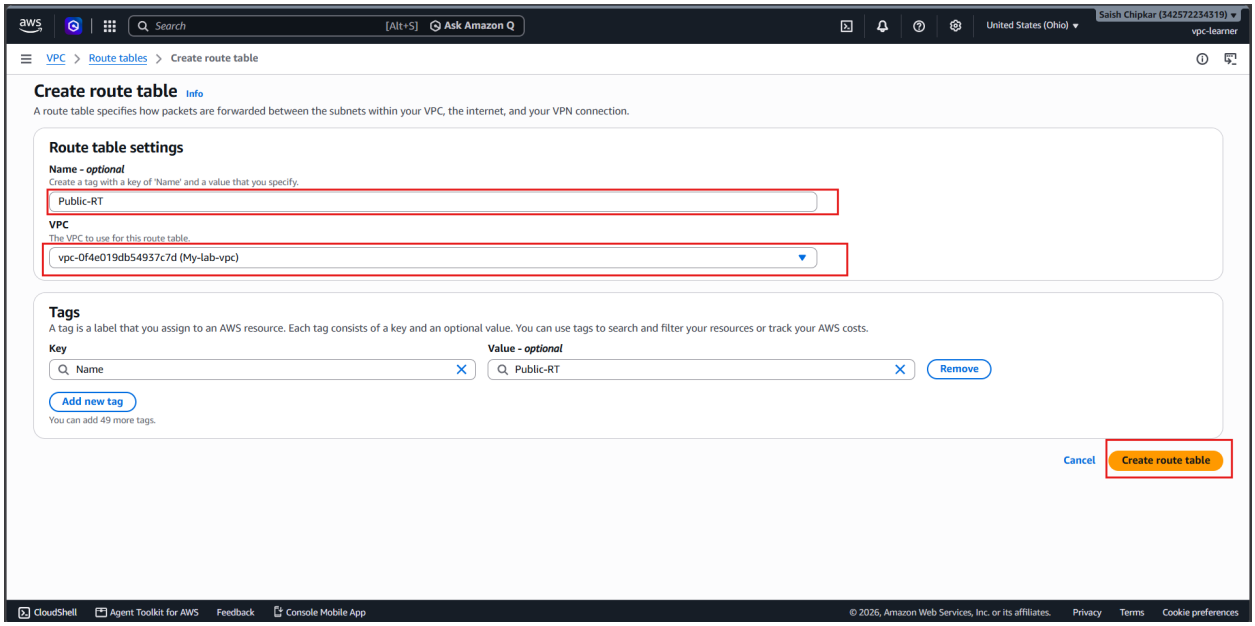


Figure 25: Enter Public-RT as the route table name and select My-lab-vpc.

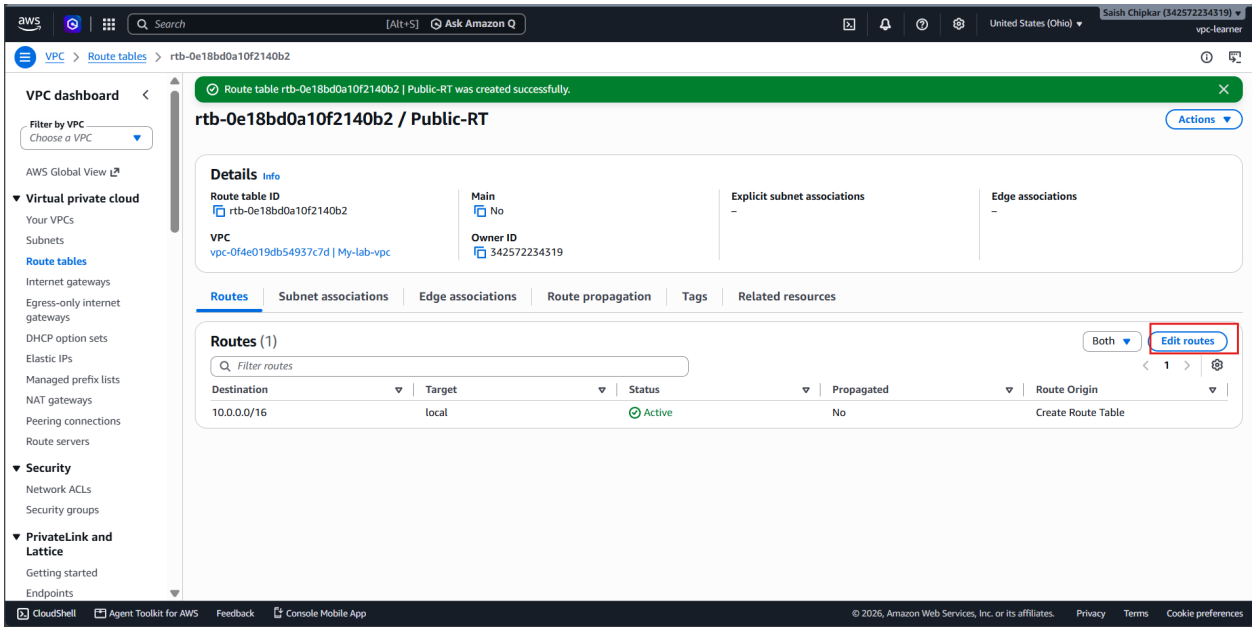


Figure 26: Public-RT created successfully with its automatically generated local route.

8.3 Add the Internet Gateway Route to Public-RT

To allow the associated subnet to route internet-bound traffic through the Internet Gateway, add a default route to Public-RT.

1. Open `Public-RT` and select the **Routes** tab.
2. Choose **Edit routes**.
3. Select **Add route**.
4. Enter `0.0.0.0/0` as the destination.
5. Select **Internet Gateway** as the target type.

6. Choose the attached gateway `My-Lab-IGW` (`igw-0be68cf9a6f122599`).

7. Choose **Save changes**.

The destination `0.0.0.0/0` represents all IPv4 destinations that do not match a more specific route. Consequently, traffic destined for the internet can be directed to the Internet Gateway.

The public route table now contains the following routes:

Destination	Target	Purpose
`10.0.0.0/16`	`local`	Routes traffic within the VPC.
`0.0.0.0/0`	`My-Lab-IGW`	Routes other IPv4 destinations through the Internet Gateway.

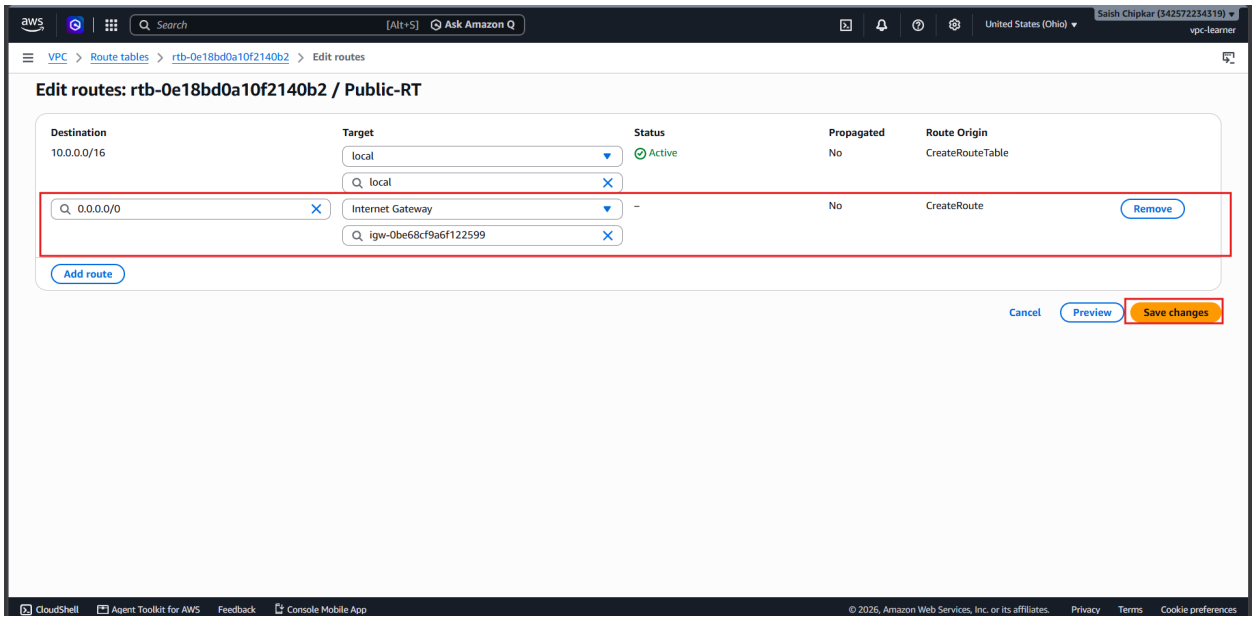


Figure 27: Add the default IPv4 route 0.0.0.0/0 and select My-Lab-IGW as its target.

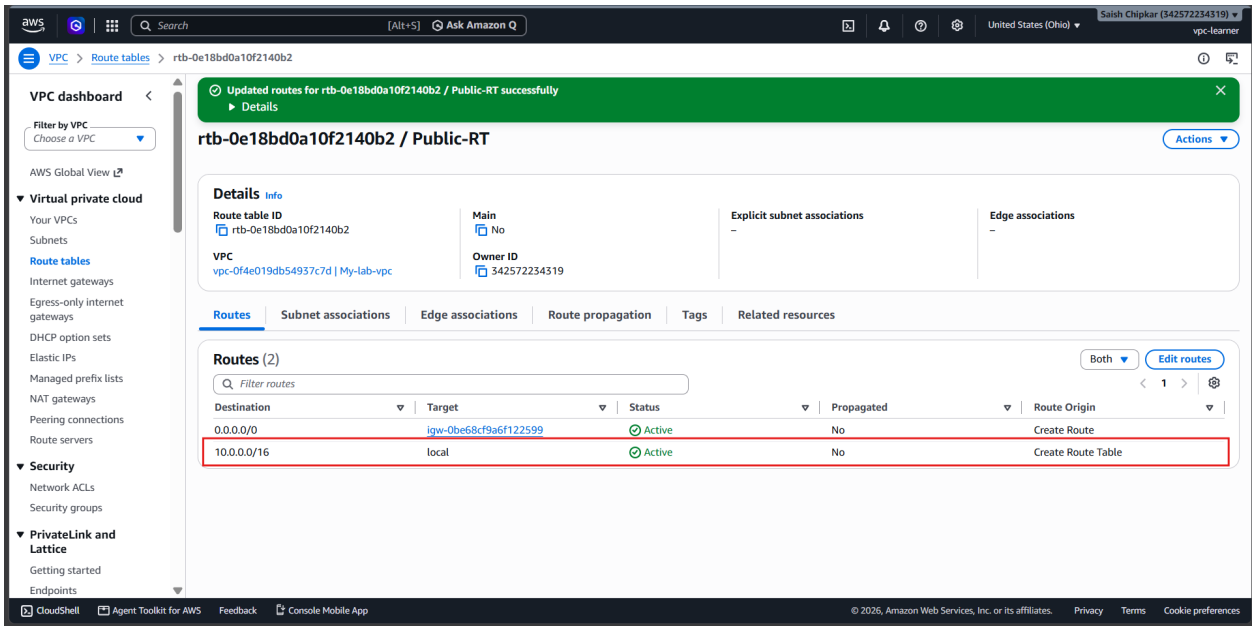


Figure 28: Public-RT showing the active local route and the active Internet Gateway route.

8.4 Create the Private Route Table

The private route table is created separately so that the private subnet does not have a direct route to the Internet Gateway.

1. Return to the **Route tables** page.
2. Choose **Create route table**.
3. Enter `Private-RT` in the Name field.
4. Select `My-lab-vpc (10.0.0.0/16)`.
5. Choose **Create route table**.

AWS automatically creates the local route `10.0.0.0/16` with target `local`. Do not add the `0.0.0.0/0` route to the Internet Gateway to this private route table.

A private subnet can still communicate with resources in the same VPC through the local route. If outbound internet access is required later, a suitable design, such as a NAT Gateway, can be configured separately.

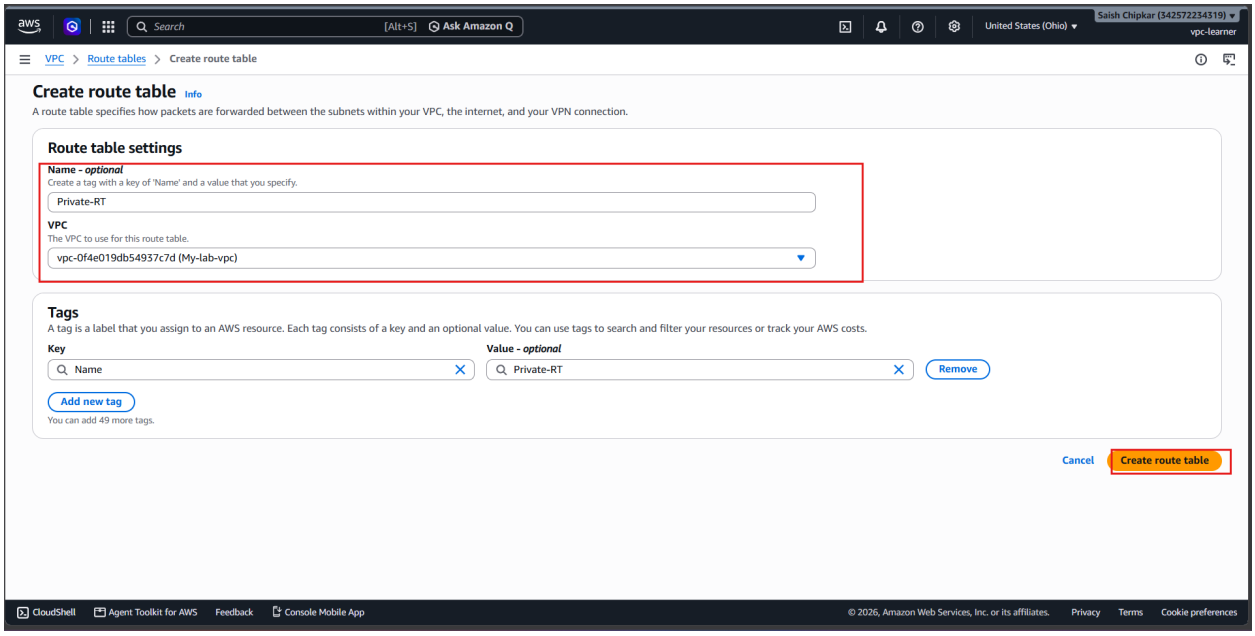


Figure 29: Enter Private-RT as the route table name and select My-lab-vpc.

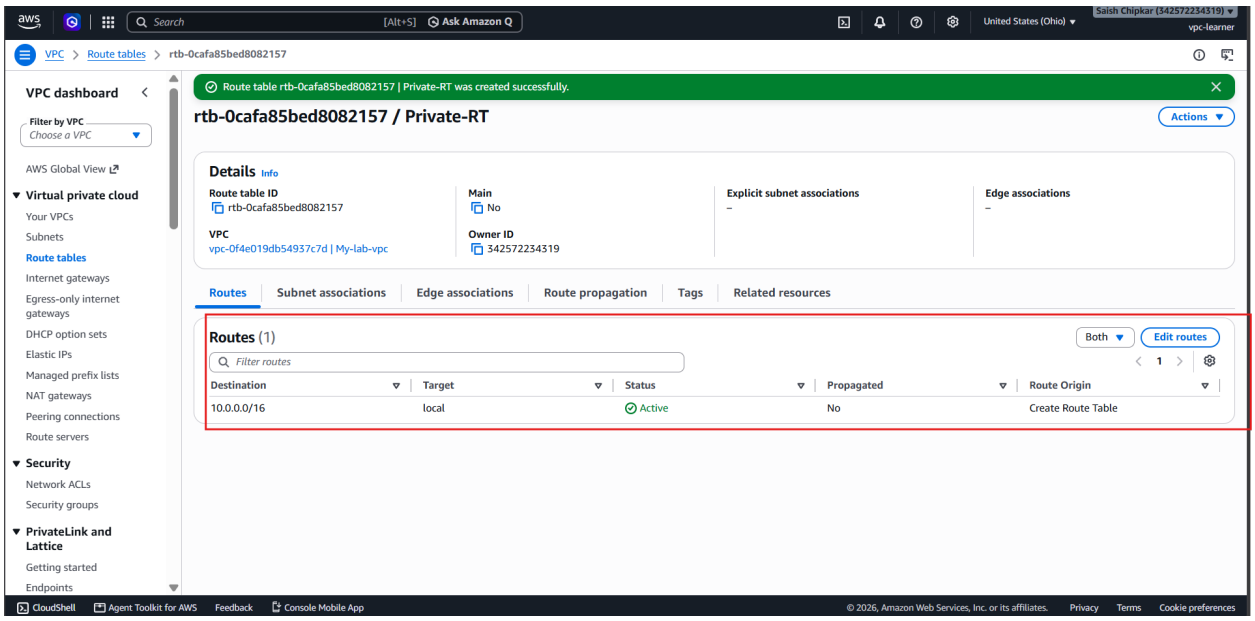


Figure 30: Private-RT created with the local route and no direct Internet Gateway route.

8.5 Associate Public-RT with Public-Subnet

Creating a route table does not automatically associate it with a subnet. The public subnet must be explicitly associated with Public-RT so that it uses the configured Internet Gateway route.

1. Open `Public-RT`.
2. Select the **Subnet associations** tab.
3. Choose **Edit subnet associations**.
4. Select `Public-Subnet`, with CIDR `10.0.0.0/24`.
5. Choose **Save associations**.

After this association, Public-Subnet uses Public-RT for its routing decisions. Its internet connectivity still depends on other required settings, including public IP addressing and security rules.

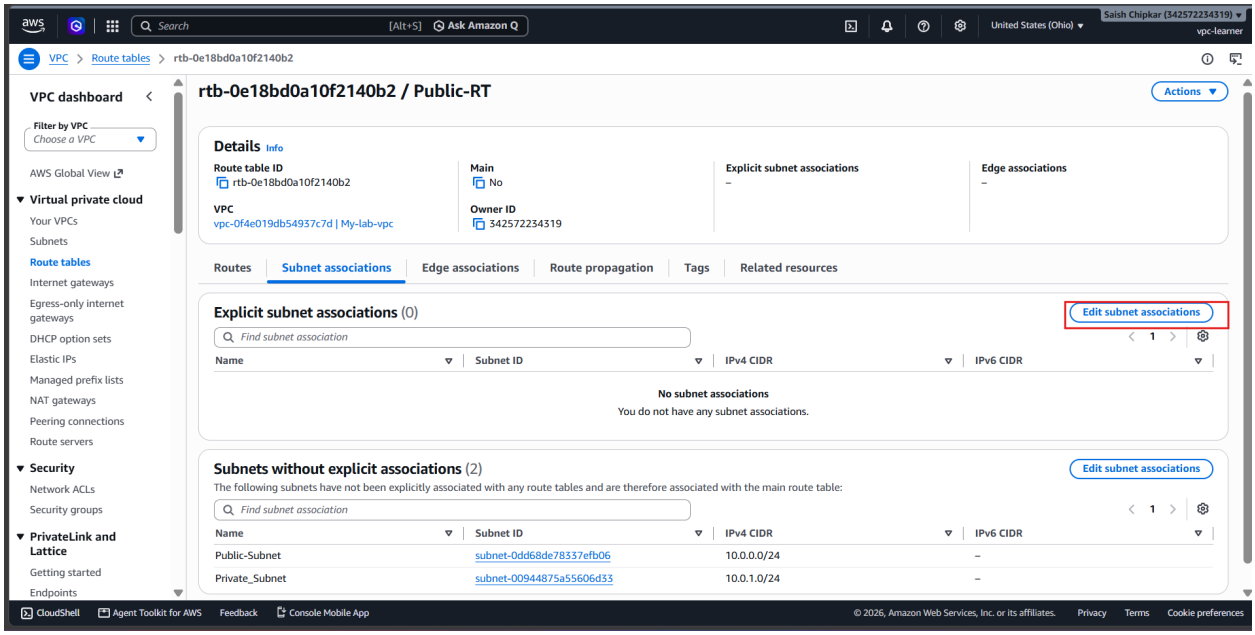


Figure 31: Open the Subnet associations tab for Public-RT.

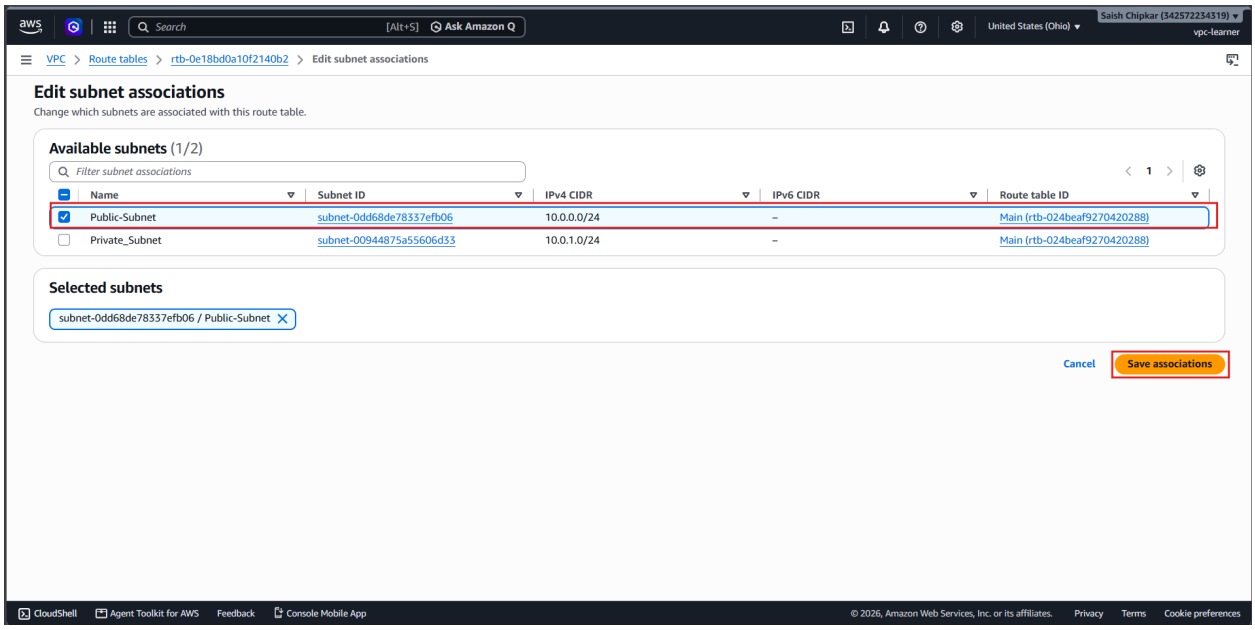


Figure 32: Select Public-Subnet with CIDR 10.0.0.0/24 for association with Public-RT.

8.6 Associate Private-RT with Private_Subnet

The private subnet must also be explicitly associated with its dedicated route table.

1. Open `Private-RT`.
2. Select the **Subnet associations** tab.
3. Choose **Edit subnet associations**.
4. Select `Private_Subnet`, with CIDR `10.0.1.0/24`.
5. Choose **Save associations**.

The private subnet now uses Private-RT. Because this route table contains only the local route, the subnet has no direct route to the Internet Gateway.

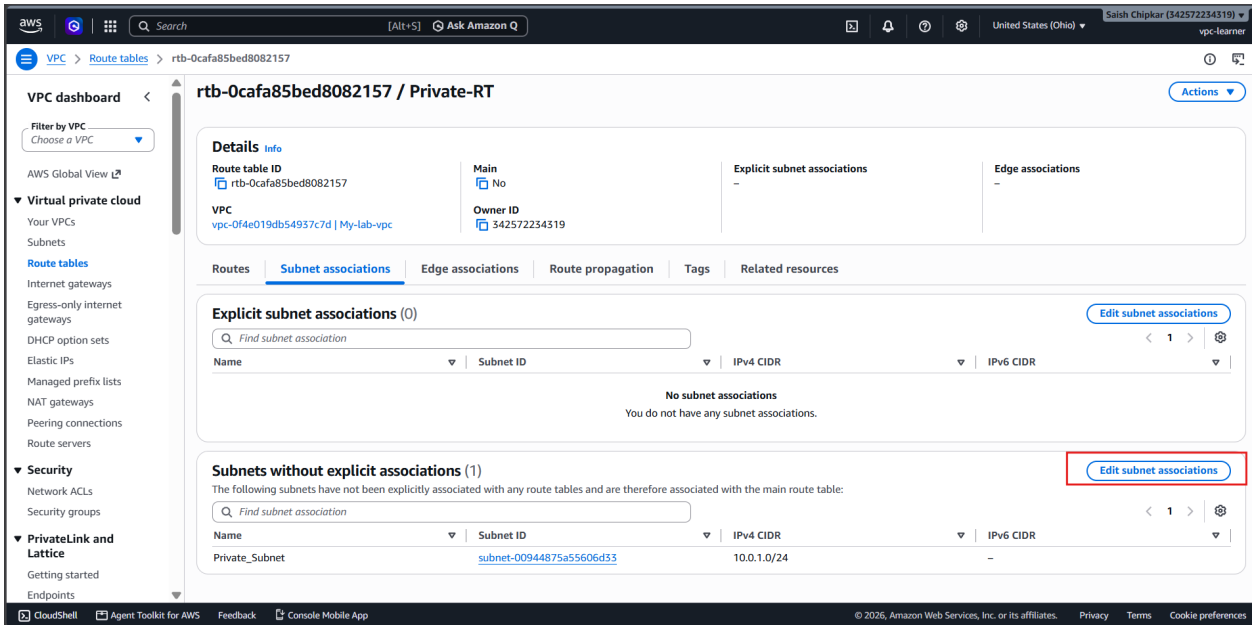


Figure 33: Open the Subnet associations tab for Private-RT.

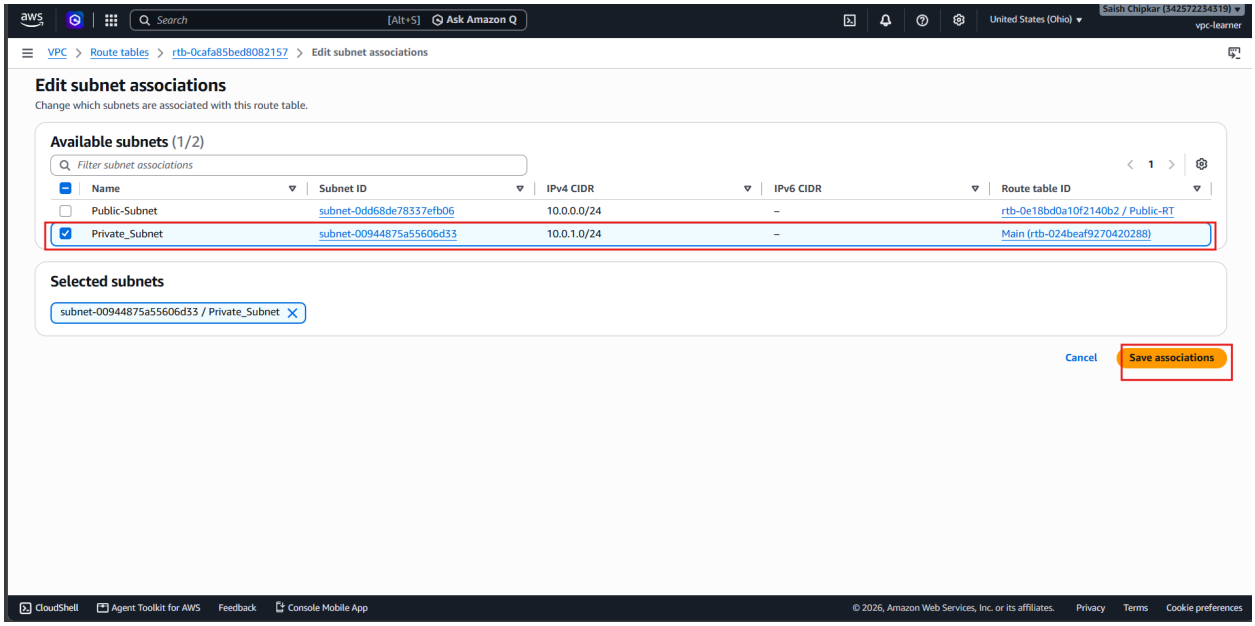


Figure 34: Select Private_Subnet with CIDR 10.0.1.0/24 for association with Private-RT.

8.7 Verify the Route Table Associations

Return to the Route Tables page and verify that both route tables are associated with the intended subnets.

Route table	Associated subnet	Routing configuration
Public-RT	Public-Subnet	Local route and Internet Gateway route
Private-RT	Private_Subnet	Local route only

The route table list should show an explicit subnet association for Public-RT and another explicit subnet association for Private-RT.

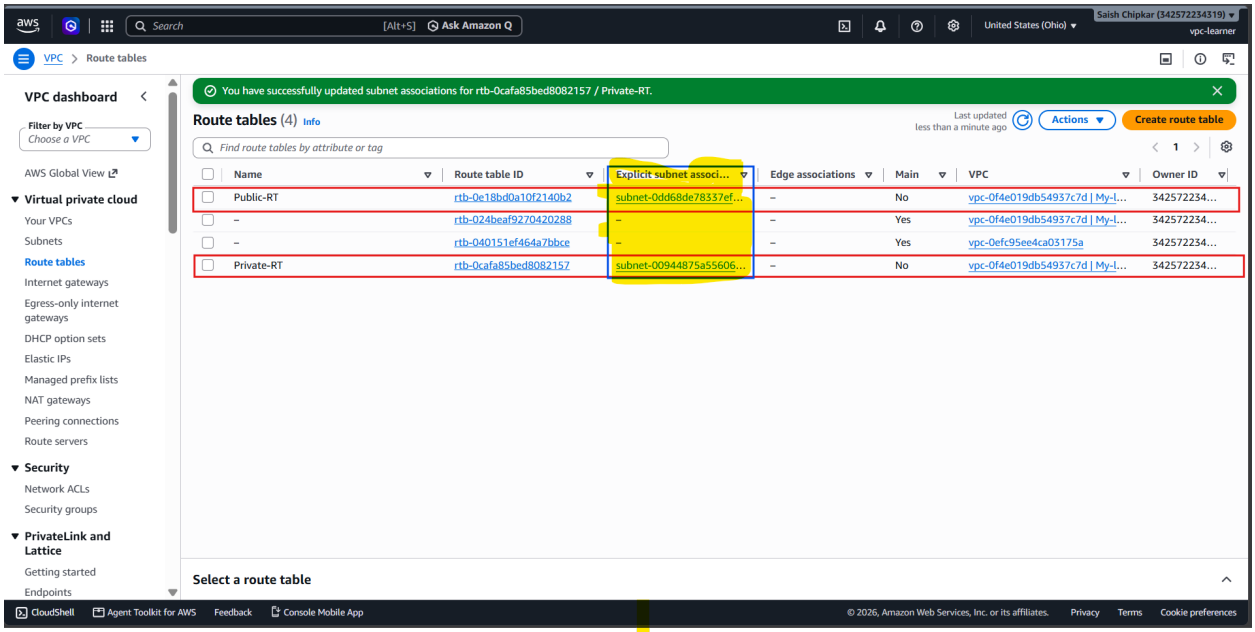


Figure 35: Route Tables page showing Public-RT and Private-RT associated with their respective subnets.

9 Step 7: Launch an EC2 Instance in the Public Subnet

Purpose: The public EC2 instance acts as a publicly reachable virtual machine and provides a jump host (bastion host) through which we can connect to the private EC2 instance.

1. Open the Amazon EC2 console and select **Instances** → **Launch instances**.
2. Enter `Public-EC2` in the Name field.
3. Select **Amazon Linux 2023** as the Amazon Machine Image (AMI).
4. Choose the `t3.micro` instance type.
5. Select the appropriate key pair for SSH access. Keep the corresponding private key file secure.
6. Under **Network settings**, choose the custom VPC `My-lab-vpc` with CIDR `10.0.0.0/16`.
7. Select `Public-Subnet`, with CIDR `10.0.0.0/24`.
8. Set **Auto-assign public IP** to **Enable**.
9. Create or select a security group that permits SSH traffic on TCP port 22 from your trusted public IP address.
10. Review the configuration and choose **Launch instance**.

Observation: The public EC2 instance was launched in the public subnet. Its private IPv4 address is `10.0.0.11`, and its public IPv4 address is `3.17.173.3`.

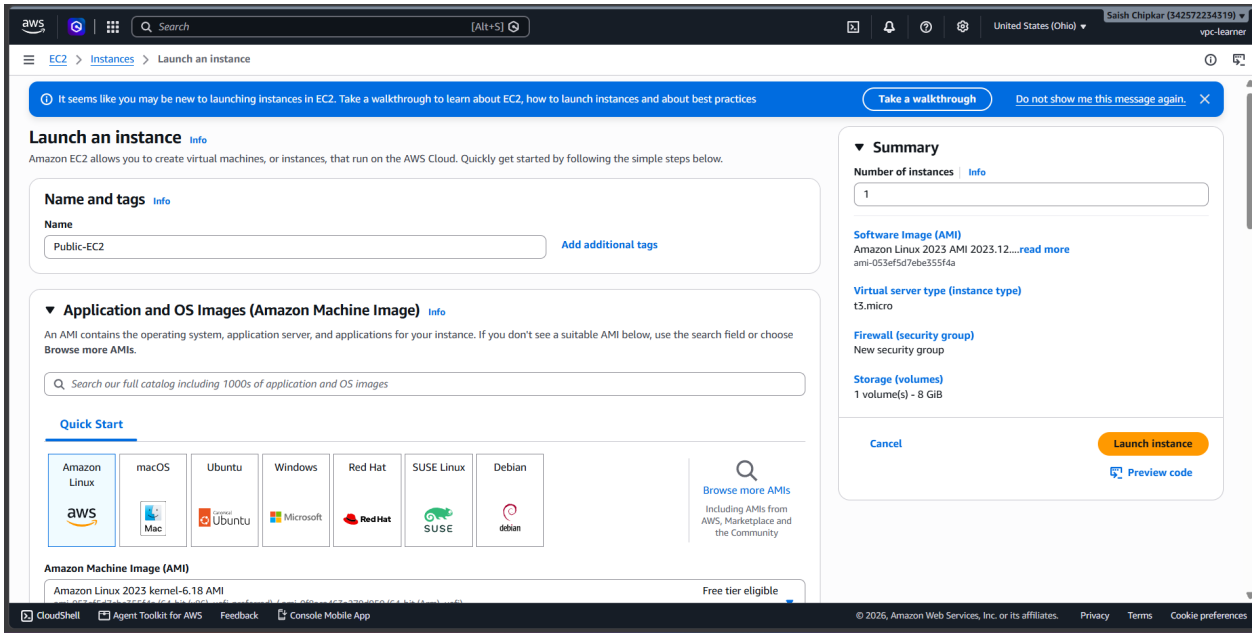


Figure 36: Launch an EC2 instance and enter the name Public-EC2.

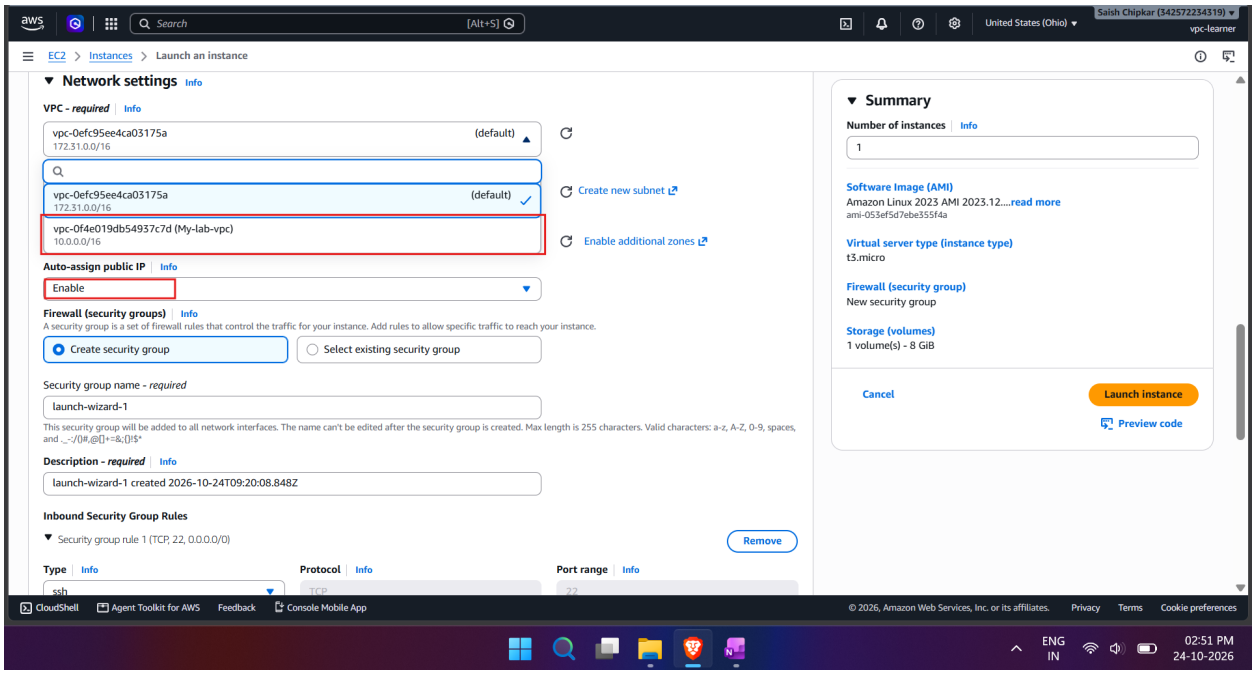


Figure 37: Select My-lab-vpc and enable automatic public IPv4 assignment.

10 Step 8: Launch an EC2 Instance in the Private Subnet

Purpose: The private EC2 instance hosts resources that should not be directly reachable from the public internet. It can be accessed through the public EC2 instance using its private IPv4 address.

1. In the EC2 console, choose **Launch instances**.
2. Enter `Private-EC2` in the Name field.
3. Select the same operating system, Amazon Linux 2023, and the `t3.micro` instance type.
4. Select the appropriate key pair and configure the instance's storage.

5. Under **Network settings**, select `My-lab-vpc`.
6. Select `Private_Subnet`, with CIDR `10.0.1.0/24`.
7. Set **Auto-assign public IP** to **Disable**. This prevents automatic assignment of a public IPv4 address to the instance.
8. Configure the security group to allow inbound SSH traffic on TCP port 22 from the public EC2 instance's security group. This is preferable to allowing SSH from `0.0.0.0/0`.
9. Review the configuration and choose **Launch instance**.

Observation: The private EC2 instance was launched in the private subnet. Its private IPv4 address is `10.0.1.197`. The instance has no public IPv4 address.

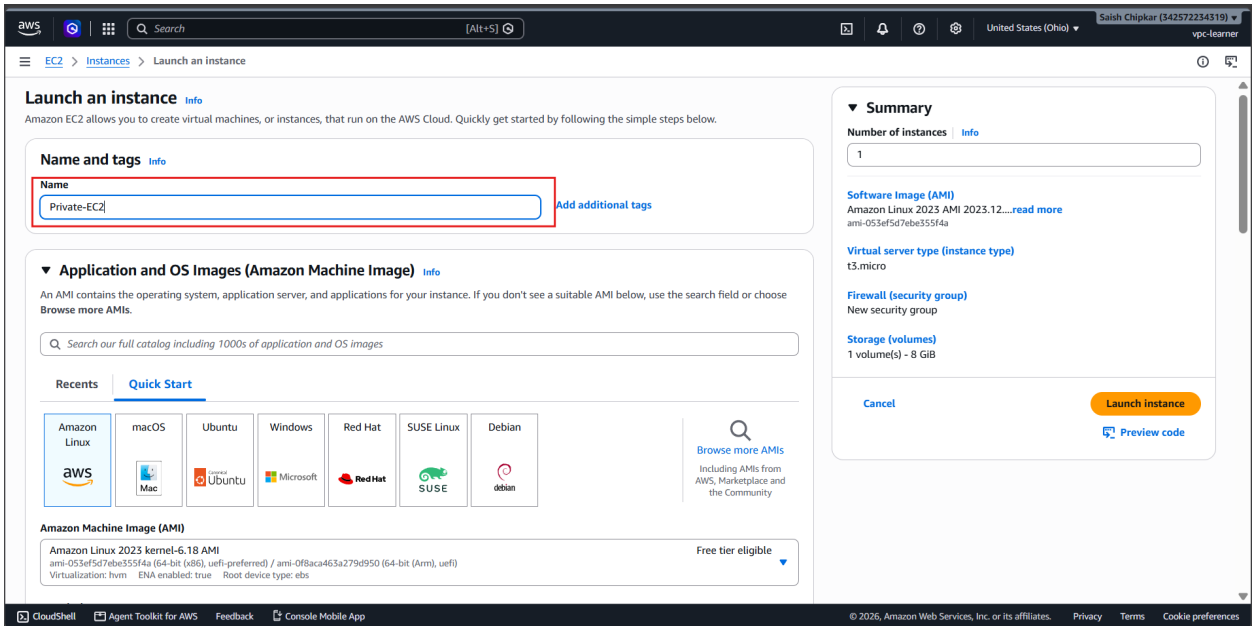


Figure 38: Enter the instance name Private-EC2.

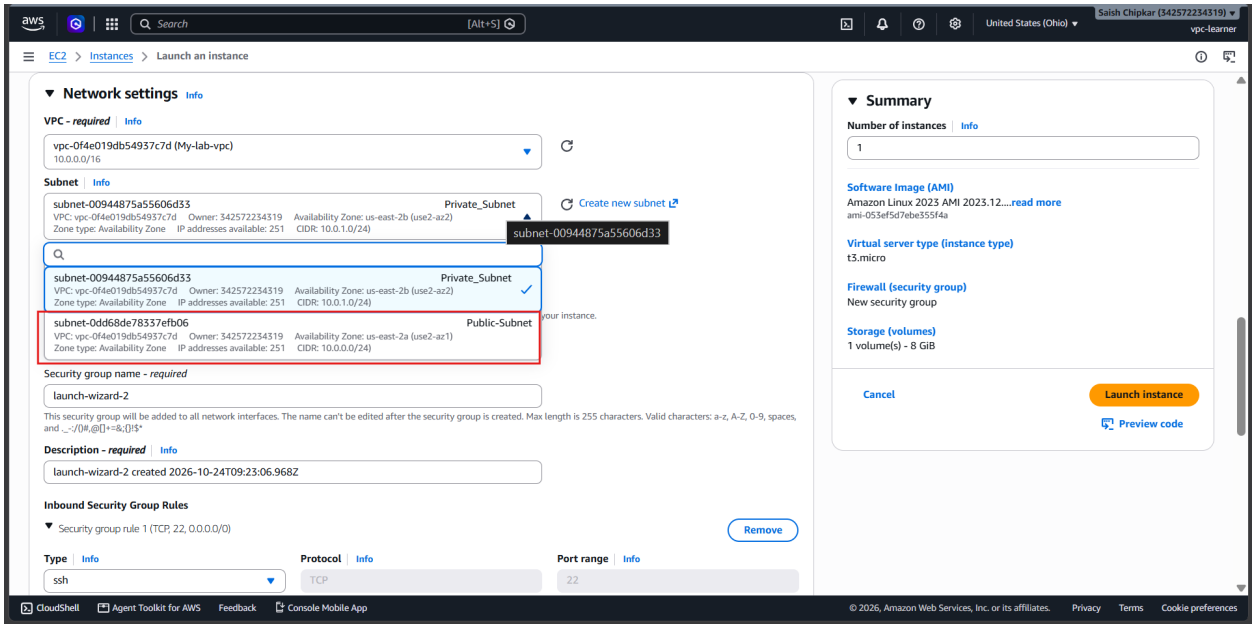


Figure 39: Select the private subnet with CIDR 10.0.1.0/24.

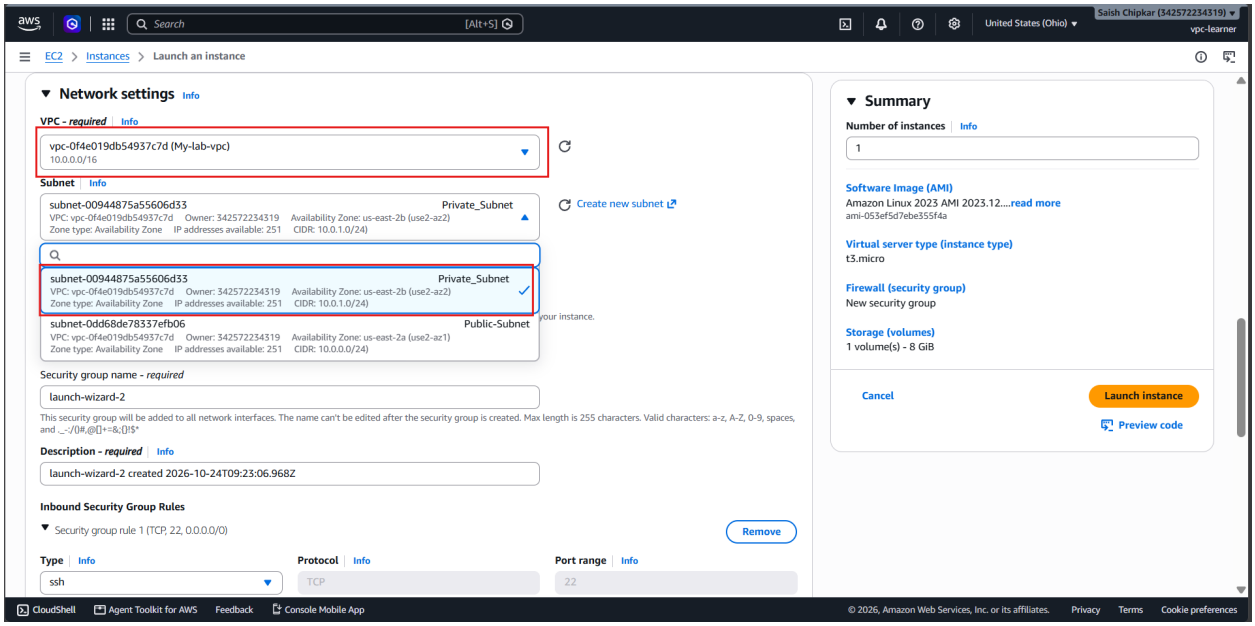


Figure 40: Disable automatic public IPv4 assignment and launch the instance.

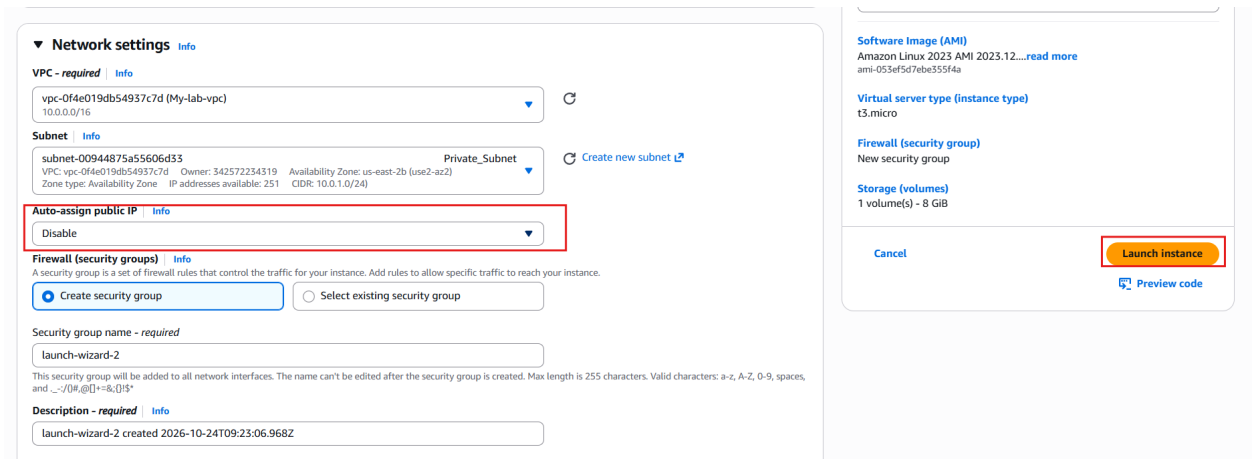


Figure 41: Verify the launched EC2 instance details.

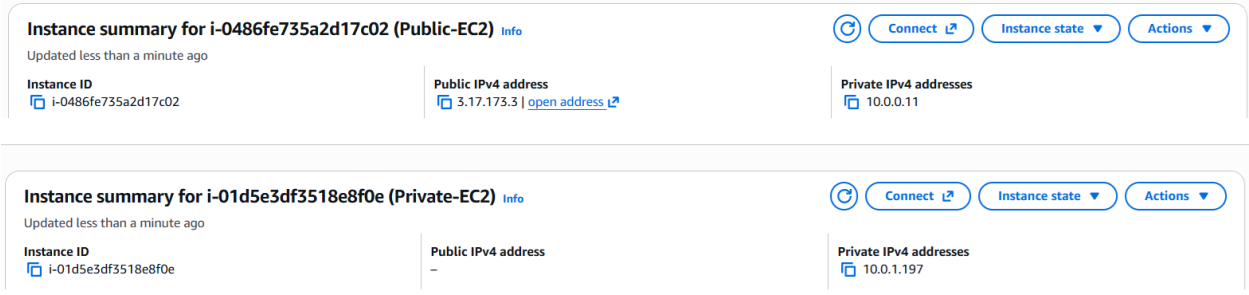


Figure 42: Public and private EC2 instances with their IPv4 addresses.

11 Step 9: Connect to the Public EC2 Instance

Purpose: Secure Shell (SSH) provides encrypted remote terminal access to a Linux EC2 instance.

1. Ensure that the public EC2 instance is running and has a public IPv4 address.

2. Verify that its security group allows inbound SSH traffic on TCP port 22 from your trusted public IP address.

3. From your local terminal, connect using the private key associated with the instance:

```
ssh -i public.pem ec2-user@3.17.173.3
```

Note: Replace the IP address with the current public IPv4 address of your instance. The key file must be accessible to your local SSH client, with appropriate file permissions.

After a successful connection, the terminal provides a shell on the public EC2 instance.

Observation: The public instance is reachable through its public IPv4 address because its network configuration, routing, and security rules permit the SSH connection.

12 Step 10: Connect to the Private EC2 Instance Using SSH

Purpose: A private EC2 instance does not require a public IPv4 address to communicate with another instance in the same VPC. The public EC2 instance can act as an intermediate host for accessing the private instance.

12.1 Prerequisites

Before connecting, verify the following:

- Both EC2 instances are running in `My-lab-vpc`.
- The public instance is in `Public-Subnet`, and the private instance is in `Private_Subnet`.
- The private instance's security group permits inbound SSH traffic from the public instance's security group.
- The public instance can access the private instance through its private IPv4 address.
- The SSH private key required for the connection is available on the public instance. For a real deployment, prefer SSH agent forwarding or another secure key-management method rather than copying a private key to a server.

12.2 SSH Connection Command

After connecting to the public EC2 instance, execute the following command in its terminal:

```
ssh -i public.pem ec2-user@10.0.1.197
```

Here:

- `ssh` initiates a secure remote connection.
- `-i public.pem` specifies the private key file used for authentication.
- `ec2-user` is the default login username for Amazon Linux.
- `10.0.1.197` is the private IPv4 address of Private-EC2.

If the connection succeeds, the terminal displays the login banner for Amazon Linux 2023 and a shell prompt belonging to the private instance.

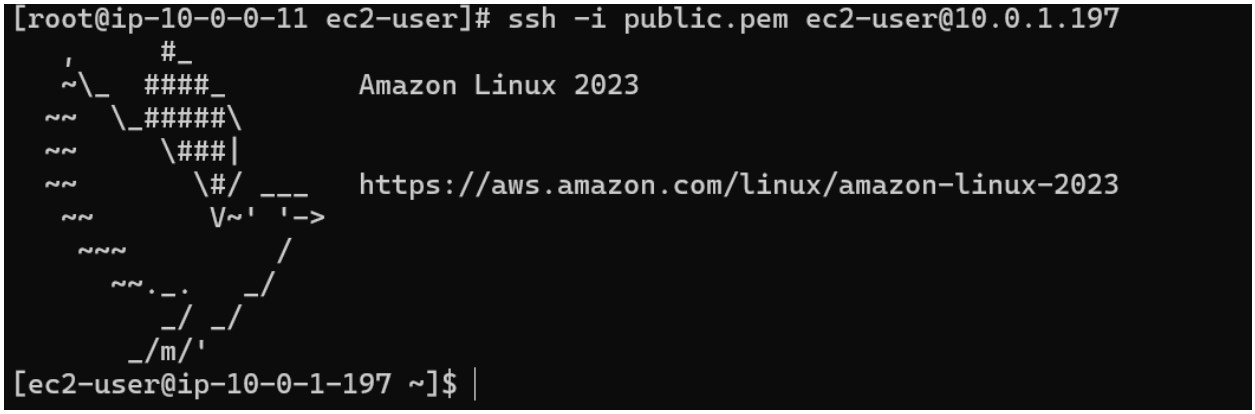


Figure 43: Successful SSH connection from Public-EC2 to Private-EC2.

Observation: The private EC2 instance was accessed through the public EC2 instance using its private IPv4 address. The private instance did not need its own public IPv4 address for this connection.

13 Step 10: Verify the EC2 Networking Configuration

The following table summarizes the EC2 instances created during the lab.

Table 1: EC2 instance configuration

Parameter	Public-EC2	Private-EC2
VPC	My-lab-vpc	My-lab-vpc
Subnet	Public-Subnet	Private_Subnet
Subnet CIDR	10.0.0.0/24	10.0.1.0/24
Private IPv4	10.0.0.11	10.0.1.197
Public IPv4	3.17.173.3	Not assigned
Instance type	t3.micro	t3.micro
Operating system	Amazon Linux 2023	Amazon Linux 2023

13.1 Results

- A public EC2 instance was created in the public subnet.
- A private EC2 instance was created in the private subnet.
- The private EC2 instance was configured without a public IPv4 address.
- SSH connectivity from the public instance to the private instance was verified using the private IP address `10.0.1.197`.
- The experiment demonstrates how a public instance can serve as a jump host for accessing a private instance within the same VPC.

14 Conclusion

This lab demonstrated the creation of a custom VPC with public and private subnets, followed by the deployment of EC2 instances in both subnets. The public EC2 instance was accessible through its

public IPv4 address, while the private EC2 instance was accessed through SSH using its private IPv4 address.

The experiment illustrates that an instance does not need a public IPv4 address to communicate privately within a VPC. Appropriate routing, security-group rules, and SSH authentication are essential for successful connectivity.